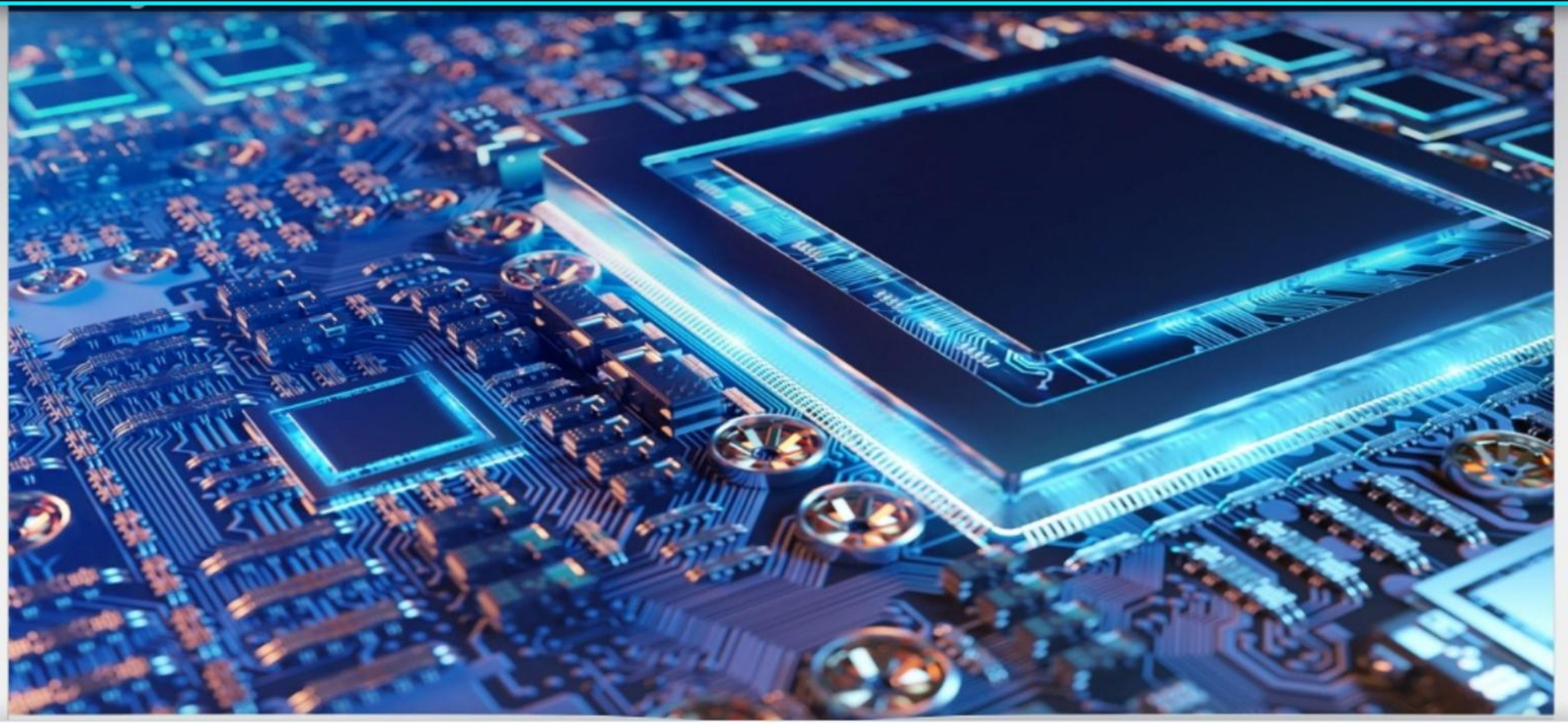
Project Sensor_STOPWATCH_UART_FIFO Feedback



2026.08.21.

작성자 : 김 경 표

Contents

Ⅰ. 공통 Feedback

Ⅱ. 레퍼런스

Ⅲ. Trouble Shooting

I

Ⅰ. 공통 Feedback

Ⅱ. 레퍼런스

Ⅲ. Trouble Shooting

I 공통 Feedback

사례1 : 개조식 표현 미작성

◆ **Feedback**

PPT, 원칙적으로 **개조식 작성**을 기본을 함

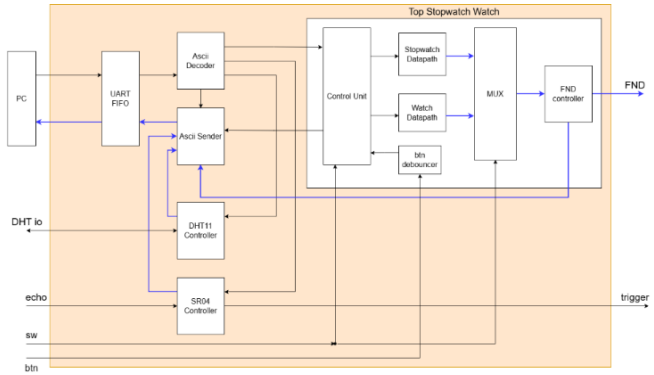
◆ **특징 및 효과**

특징 : 서술어 어미 종결형, 명사형으로 끝맺음

효과 : 불필요한 접속사 미사여구 생략 → 가독성↑

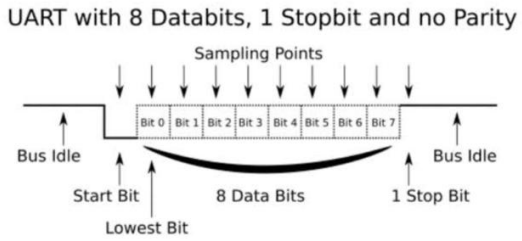
Block Diagram

❑ Block Diagram
- Stopwatch_Watch에 센서, UART, Ascii Decoder / Sender 추가
- 파란선이 Datapath 검은선이 제어신호이다.



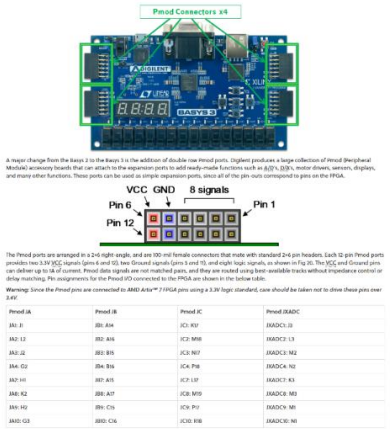
Description of I/O Used

❑ UART 통신
- 통신 방법 중 하나로 주로 interface에서 사용
- clk이 존재하지 않는 통신
- 단방향 전달을 위해 1bit의 line이 필요하다



Description of I/O Used

❑ Basys 3 보드 Pmod Connector
- Basys 3 보드에는 4개의 Pmod Connector 존재
- 외부 I/O 연결을 돕는다.
- 하나의 Pmod Connector에는 2개의 Vcc Pin, 2개의 Gnd Pin, 8개의 data Pin이 존재



대한상공회의소
서울기술교육센터

I 공통 Feedback

사례2 : 공백의 의미

◆ **Feedback**

공백 : 자료 작성중인 상태로 인식할 가능성 ↑

슬라이드 내용과 적합한 이미지 및 내용 삽입

단, Fully하게 채우지 말 것

프로젝트 개요

개요 설명

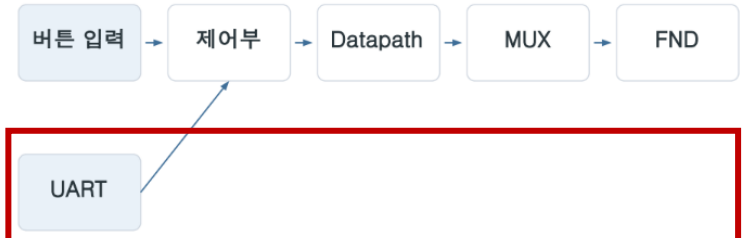
stopwatch와 watch의 시간을 segment에 표시하고, 버튼과 UART 모듈을 통해 제어한다.

SR04, DHT11 sensor를 이용하여 센서 값 또한 segment에 표시한다.

UART_fifo_loopback을 사용하여 디버깅에 용이하도록 PC에서 전송한 제어 신호를 볼 수 있도록 한다.

Ascii decoder를 통해 제어신호를 생성하며 /get 명령어를 통해 FPGA에서 생성된 데이터를 PC로 송신한다.

온도, 습도의 경우 정수 부분을 FND에 표시한다.



Case 1)

적절한 순서도 or Block Diagram

삽입하여 완성된 형태로 작성

개발 환경 및 사용 장비

개발 환경

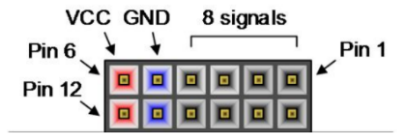
- 툴 : Vivado 2020.2
- 언어 : Verilog HDL
- 보드 : Basys3 Artix-7 FPGA Board
- sensor : SR04, DHT11
- clock : 100 MHz System Clock
- 통신 : UART , comportmaster

Case 2)

적절한 이미지 삽입 or

분할 작성으로 공백 최소화

Basys3 사용 자원(Ports)



JA[1] = echo
JA[2] = trigger
JA[3] = dht_io

Case 3)

적절한 보드 이미지 삽입 or

장표를 통한 Ports 설명으로 공백 최소화

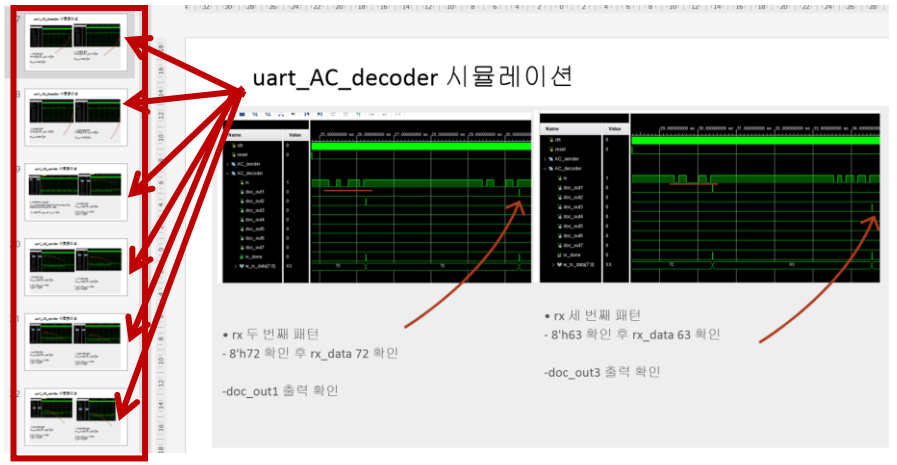
I 공통 Feedback

사례3 : 슬라이드 취합 및 내용 병합

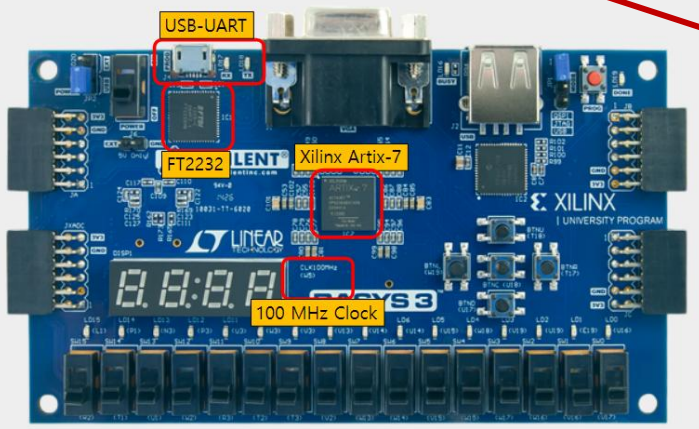
◆ **Feedback**

동일 제목에 슬라이드 구성 多 → 단순 내용 나열 (장황함)↑

Numbering or 핵심 내용만 간략하게 기술할 것



보드 설명



Basys3 주요 구성

•Xilinx Artix-7 FPGA : Verilog로 설계한 전체 디지털 회로 구현

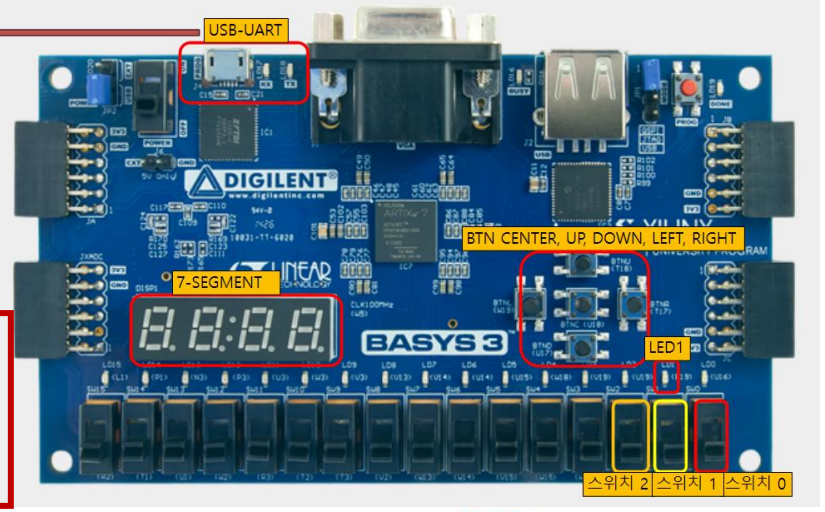
•100 MHz Clock : 시스템 기준 클럭

•FT2232 : USB-JTAG 및 USB-UART 통신 지원

•USB-UART : PC와 FPGA 사이 UART 데이터 송·수신

보드 설명





취합 및 통합

1장으로 정리

I 공통 Feedback

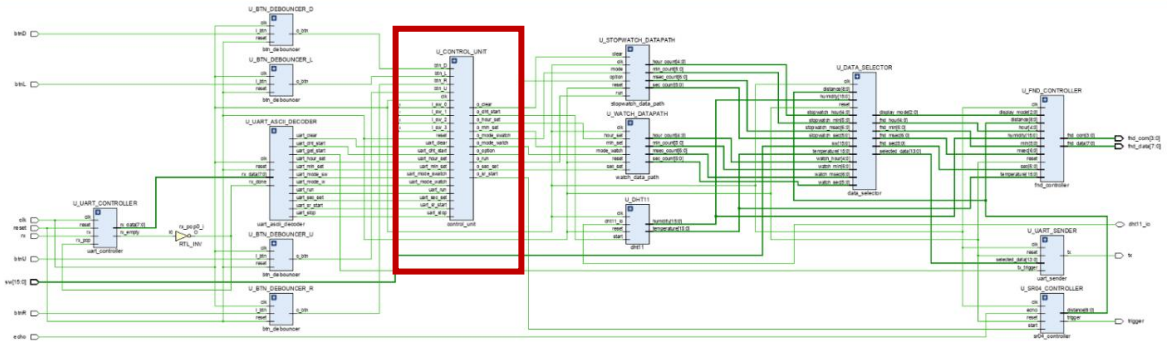
사례4 : Text 내용 vs Image (Sch. or Timing Diagram) Mapping X

◆ **Feedback**

순서 or 해당 Block 미표기, Timing Diagram (Text 설명 부분) → 가시성 및 가독성↓

Text 내용과 Matching하여 표시 & 핵심 내용 기입 권장

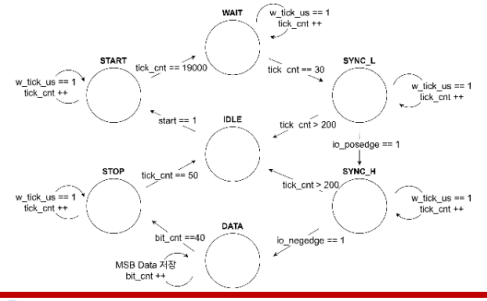
Top Module Schematic

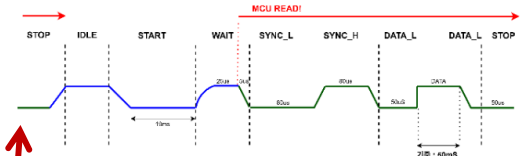


- UART RX 데이터는 UART Controller → ASCII Decoder → Control Unit 순서로 전달되어 명령별 제어 신호로 변환
- 버튼 입력은 Button Debouncer를 거쳐 안정적인 제어 신호로 변환되며, UART 명령과 함께 Control Unit로 출력
- Control Unit은 시계, 스톱워치, DHT11, SR04 모듈 start 신호를 전달
- 각 Datapath와 센서의 출력은 Data Selector에서 선택되어 FND Controller와 UART Sender로 전달

Sub Module Block Diagram

Sensor DHT11





단순 나열 지양 & 핵심이 되는 부분 표시

→Timing Diagram Status Matching을 통하여 설명하는게 효과적

- start 신호가 입력되면 약 19ms 동안 LOW로 유지한 뒤 HIGH로 전환하고, 30μs 동안 DHT11 신호 대기함
- DHT11이 약 80μs LOW와 HIGH의 SYNC 신호를 출력하면 edge를 감지하여 SYNC_L → SYNC_H → DATA 상태로 전환
- DATA 상태에서는 HIGH 유지시간을 측정하여 40μs 미만은 0, 40μs 이상은 1, negedge마다 40비트를 Shift Register에 저장
- 40비트 수신 완료 후 sum을 비교하여 습도와 온도를 출력하고, STOP 상태에서 50μs 대기한 뒤 IDLE 상태로 복귀함

I 공통 Feedback

사례5 : 단순 Text 서술 지양

◆ **Feedback**

Text 비중이 높을수록 (장황) → 가시성 및 가독성 저하 / Highlight 多 → 논점이 흐려짐

Case 1) Text 보단, **장표로 정리** 작성 → 가시성 및 가독성↑

Case 2) **필요 부분에만 Highlight 표시** → 개연성 및 전달력↑

Case1)

Operational Functions

<Command List>

- /stwa : stopwatch 모드로 전환 및 현재 스탑워치 PC로 출력
- /time : watch 모드로 전환 및 현재 시간 PC로 출력
- /dist : SR04 모드로 전환 및 현재 측정 거리 PC로 출력
- /humi : DHT11 모드로 전환 및 현재 측정 습도 PC로 출력
- /temp : DHT11 모드로 전환 및 현재 측정 온도 PC로 출력

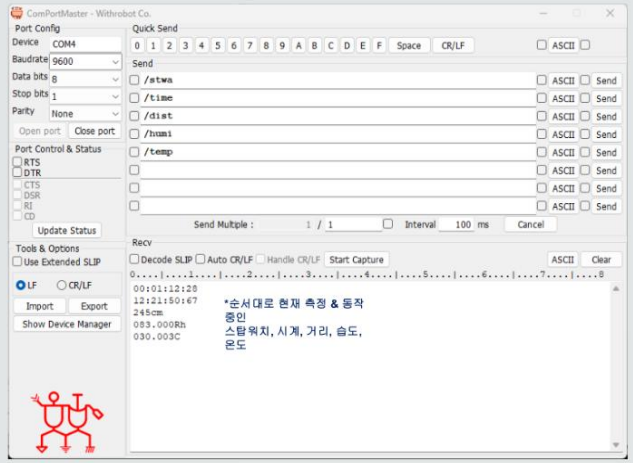
- r(run), s(stop), c(clear), m(mode) : stopwatch 제어
- S, M, H, U, D, L, R : watch 시간 수정 제어
- T : FND 출력 자리 변경(시분/초밀리초, 온도/습도 등 전환)
- E : SR04, DHT11의 센서 값 측정 실행

<Button & Switch>

- Stopwatch : btn_L(run&stop), U(mode change), R(clear), D(save time), sw[7:3] (choose save slot)
- Watch : btn_L(hour up), U(min up), R(sec up)
- SR04, DHT11 : btn_U (measure)

"<->" : toggle by switch
"||" : selected by sw[2]

- Sw[2] : stopwatch/watch <-> sr04/dht11 swap
- Sw[1] : stwa<->watch || sr04<->dht11 swap
- Sw[0] : HM<->SMSEC || dht11(humi) <-> dht11(temp) swap



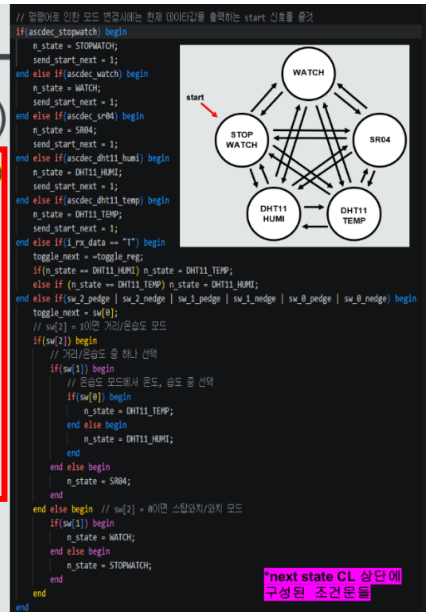
Case2)

Main Control Unit (상태 천이 원리)

- 5개 모드에 맞는 고유의 상태 존재 (stopwatch, watch, sr04, dht11(humi), dht11(temp))
- switch 혹은 pc 명령어를 통해 동일한 레벨의 천이가 발생되어야 함
- 즉 , 먼저 입력이 들어온 쪽이 모드 변경 권한을 갖고 변경시킴
- 이후 들어오는 입력에서도 항상 먼저 들어온 쪽이 권한을 가짐

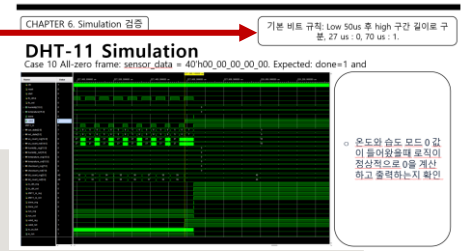
- 천이 조건으로 물리적 switch조작과 pc 명령어가 존재
- Pc 명령어는 입력 즉시 해당 상태로 강제 천이(switch 레벨 상관 X)
- Switch는 조작을 통해 레벨을 바꾸게 될 시 발생되는 tick을 이용해 현재의 switch 레벨을 전부 감지하여 올바른 상태(모드)로 천이
- Switch의 tick과 level을 동시에 감지하는 원리를 통해 PC 명령어를 이용한 강제 상태 천이가 발생한 이후에도 switch 조작이 먹통 되지 않고 적절한 상태(모드)로 정상 천이가 가능해짐

- 또한 sw[0]의 기능인 시분/초밀리초 전환, 습도/온도 전환을 명령어로도 제어하기위해 별도의 toggle신호를 통해 sw[0]의 물리적, 소프트웨어적 동시에 제어하도록 구현



I 공통 Feedback

사례6 : 미괄식 자료 작성

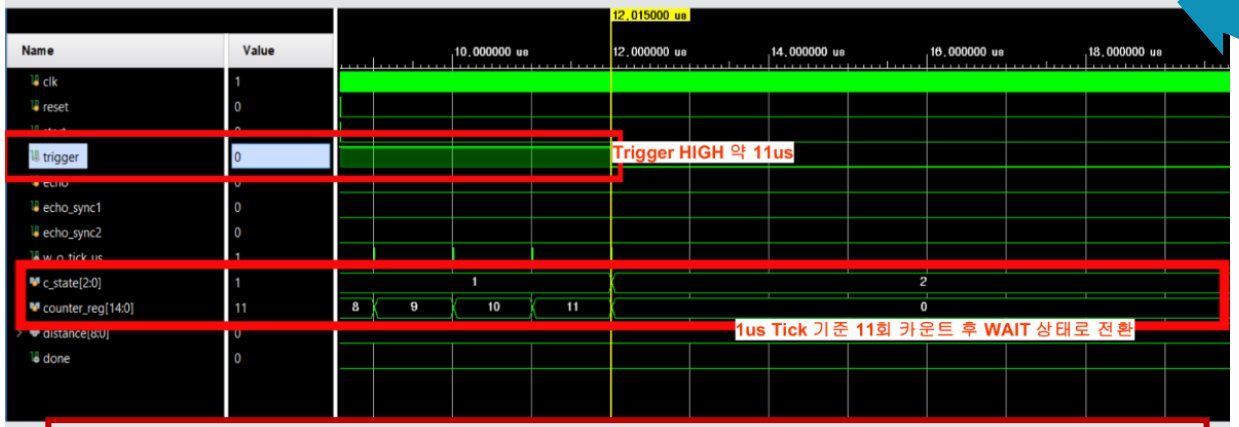


◆ **Feedback**

두괄식(상단) : **슬라이드 내용을 요약 (결론)** → 가시성 및 가독성↑ ex) 검증 결과 : 정상 동작함

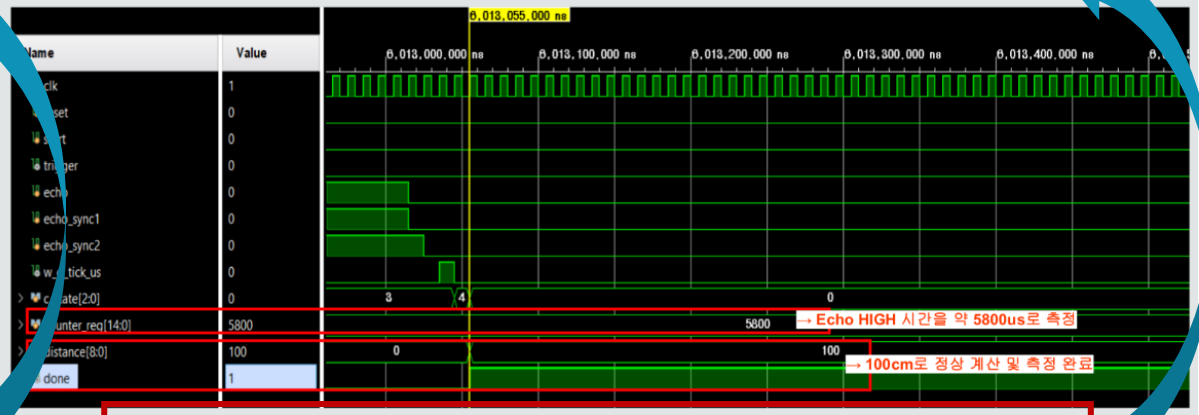
미괄식(하단) : 기타 주석 및 보강 내용 (Text는 작게) ex) Note. 5800 / 58 = 100cm

SR04 Simulation (Trigger 신호 생성 검증)



Start 입력 후 START 상태에서 1us Tick을 기준으로 카운터를 증가시키며 Trigger 신호를 약 11us 동안 HIGH로 유지한 후 WAIT 상태로 전환되는 것을 확인하였다.

SR04 Simulation (거리 계산 검증) (100cm)



Echo HIGH 시간을 약 5800us로 측정하고, 5800 / 58 = 100cm의 거리값을 계산하였다.
이후 distance = 100, done = 1로 출력되어 100cm 거리 측정이 완료됨을 확인하였다.

I 공통 Feedback

사례7 : Block Diagram 단순 언급

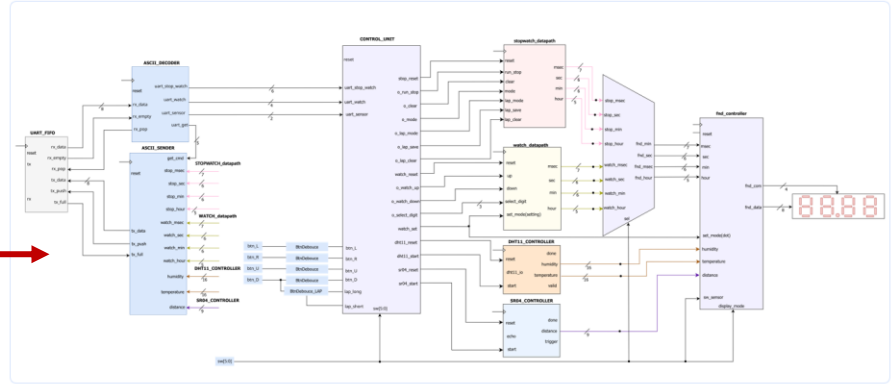
◆ **Feedback**

단순 Topology 및 구조 제시 : **적절함**

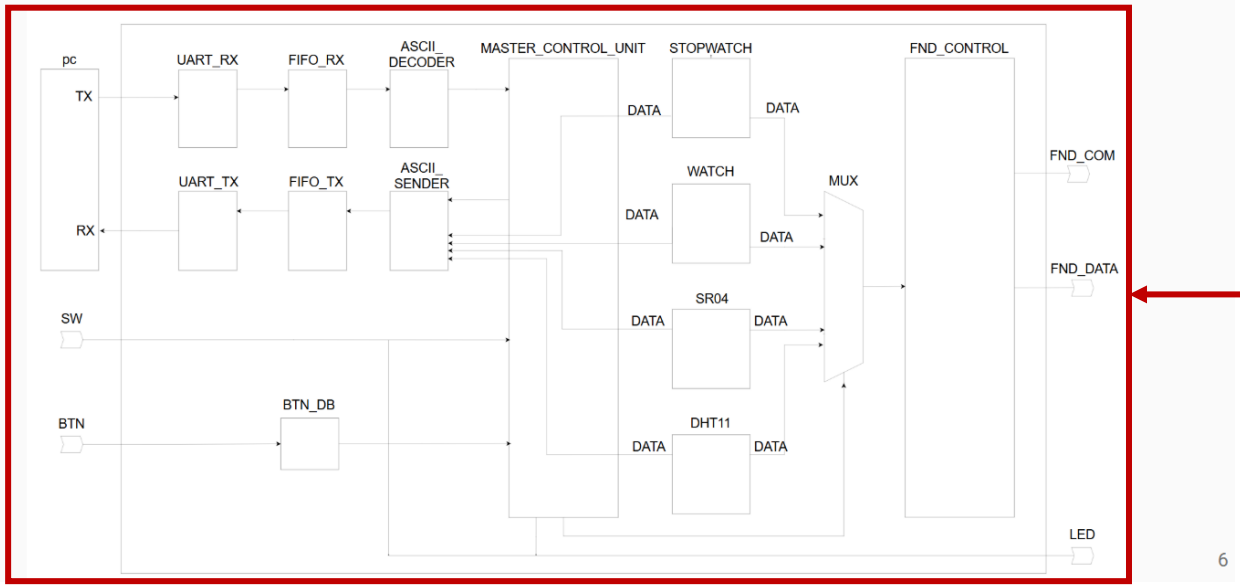
But, Core IP 및 시스템 흐름 설명 : **표시 및 Labeling 필요**

12 TOP

전체 Module Block Diagram

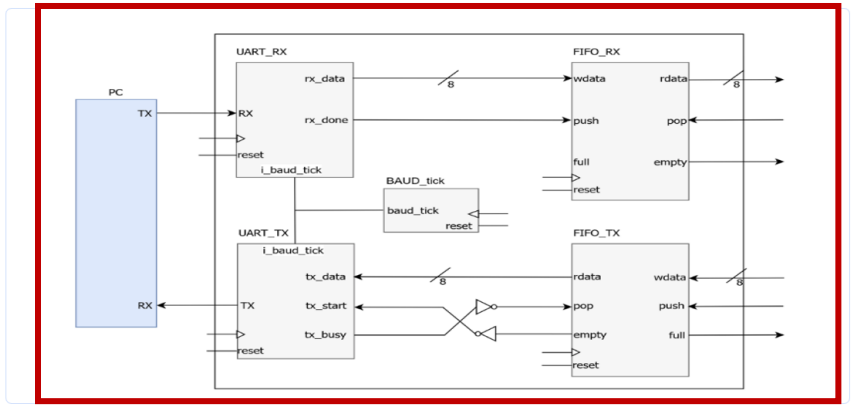


Sensor_UART_FIFO_SW_W Block Diagram



03 UART FIFO

UART RX/TX + FIFO 구조



대한상공회의소 서울기술교육센터

I 공통 Feedback

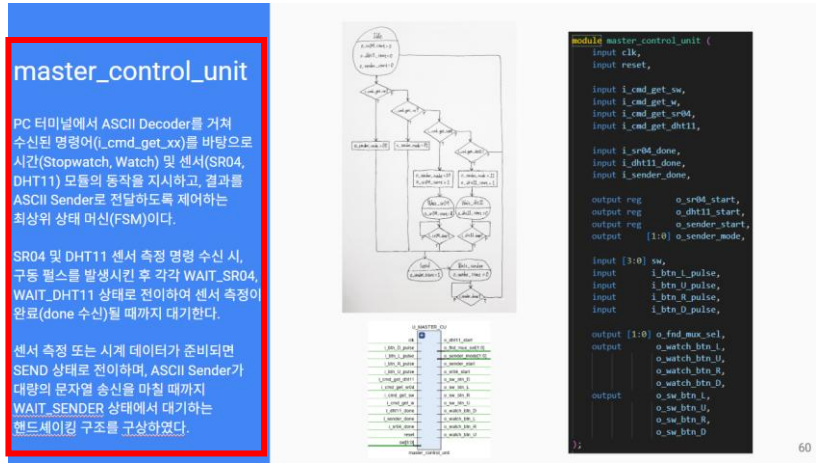
사례8 : Text 연결 및 Pointer Mismatch

◆ **Feedback**

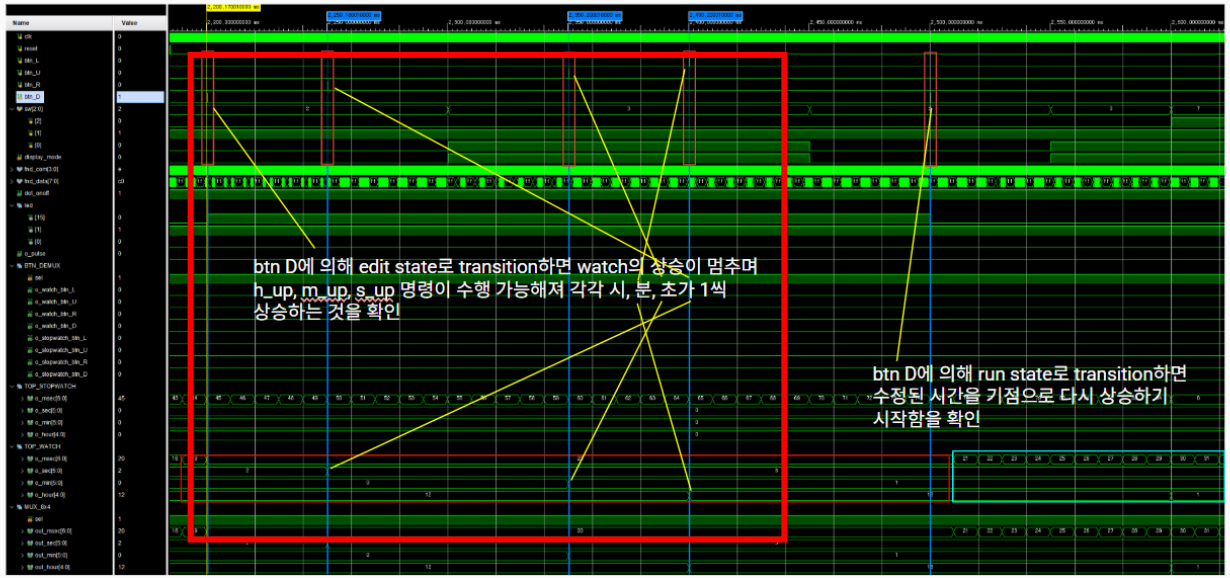
현재 자료) **복잡성↑, 가독성↓**

Text와 Pointer Mismatch : **해당 항목별로 범례 필요** (시,분,초)

내용과 Pointer가 일치 : **가독성, 가시성↑**



Simulation (edit state - h, m, s up)



Simulation (24h↔12h change)



I 공통 Feedback

사례9 : 결론 및 고찰

◆ **Feedback**

단순 검증 및 구현 (by Text 나열) → What? How? 내용 누락↑

결론 : 설계 및 검증을 통한 **결과를 논하는 과정, 정상 동작 여부 및 수치 기입**

고찰 : **현 시점 Issue** or **향후 Develop** 내용 작성

결론 및 고찰

결론 내용?

고찰 내용?

→ **분류 필요**

다수의 하위 모듈(센서, 타이머, FND, UART)을 하나의 Top 모듈로 결합한 최상위 통합 시스템의 시뮬레이션을 성공적으로 수행하고 모든 시나리오에 대한 기능적 무결성을 검증하였다.

FIFO를 활용한 비동기 백프레셔 제어를 통해 고속 데이터 포맷팅 모듈과 저속 UART 송신 모듈 간의 속도 차이로 인한 통신 병목을 완벽하게 해결하였다.

Inout 핀의 양방향 제어 및 디바운싱, 하드웨어 MUX 라우팅 차단 기술을 적용하여 전기적 충돌과 모드 간 논리적 간섭을 원천적으로 방지하는 방어형(Defensive) 아키텍처를 구현하였다.

궁극적으로 보드의 물리적 스위치/버튼 조작과 PC 기반의 ASCII 원격 제어 명령이 단일 하드웨어 플랫폼 내에서 공존하는 견고한 임베디드 제어 시스템을 완성할 수 있었다.

향후 본 설계 아키텍처를 확장하여 다중 센서 기반의 IoT 엣지 디바이스나 스마트 홈 원격 모니터링 시스템 등 보다 복잡한 실무 응용 분야로의 적용을 고려할 수 있다.

79

04. 결론

- FIFO data를 PC와 UART 통신으로 주고 받는 과정을 이해 할 수 있다.
- Simulation 을 통해 FIFO memory 의 동작을 검증하였다.

만약 못했다면?
어떻게 검증?
왜 그렇게 검증?
동작 특성은?

I 공통 Feedback

사례10 : 결론이 없는 슬라이드

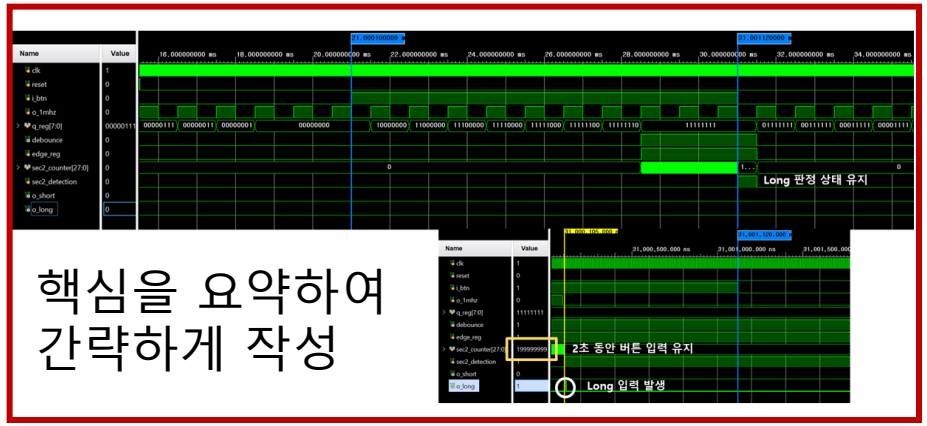
◆ **Feedback**

결론이 없다 → **내용 전달 X (특히 Sim. Result)**

Title에 맞게 간략하게 기술하여야 함

05 main_control_unit

btn_debounce_lap Long

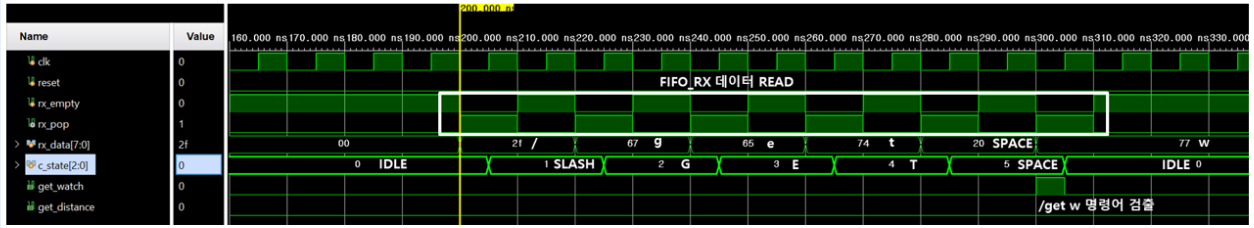


핵심을 요약하여
간략하게 작성

04 ASCII Decoder

"/get w" FIFO_RX READ 파형

결론?

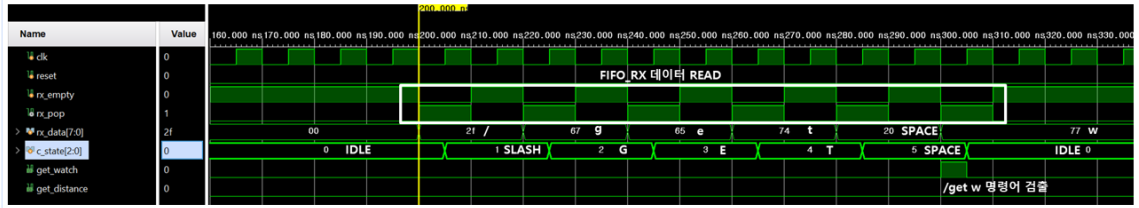


'/','g','e','t',' ','w' 6글자가 순차적으로 rx_pop되며 IDLE→SLASH→G→E→T→SPACE→IDLE 전이, 마지막에 get_watch=1 pulse

04 ASCII Decoder

"/get w" FIFO_RX READ 파형

결론?



'/','g','e','t',' ','w' 6글자가 순차적으로 rx_pop되며 IDLE→SLASH→G→E→T→SPACE→IDLE 전이, 마지막에 get_watch=1 pulse

I 공통 Feedback

사례11 : 제목과 무관한 내용 구성

◆ **Feedback**

맥락 없는 내용 & 용어 오용 → **의문을 줄 가능성 多**

Slide 재구성 or 용어 뜻 알고 사용 권장

I-1 top.v — 7개 인스턴스가 곧 시스템 블록

top 은 로직을 만들지 않고 7개 모듈의 포트만 연결한다

???????????

```
//================================
btn_unit #(
    .SAMPLE_COUNT(DEBOUNCE_CNT)
) U_BTN_UNIT (
    .clk    (clk),
    .reset  (w_reset),
    .i_btn_l(btn_L),
    .i_btn_r(btn_R),
    .i_btn_u(btn_U),
    .i_btn_d(btn_D),
//================================
uart_comm #(
    .SYS_CLK    (SYS_CLK),
    .BAUD       (BAUD),
    .SEND_PERIOD(SEND_PERIOD)
) U_UART_COMM (
    .clk  (clk),
    .reset(w_reset),

    .rx(rx),
```

```
//================================
main_control_unit U_CONTROL_UNIT (
    .clk          (clk),
    .reset        (w_reset),
    .sw_mode      (sw[3:1]),
    .sw_disp      (sw[0]),
    .btn_l        (w_btn_l),
    .btn_r        (w_btn_r),
    .btn_u        (w_btn_u),
    .btn_d        (w_btn_d),
//================================
time_datapath #(
    .TICK_100HZ(TICK_100HZ)
) U_TIME_DATAPATH (
    .clk          (clk),
    .reset        (w_reset),
    .mode_sel     (w_mode_sel),
    .sw_run_stop  (w_sw_run_stop),
    .sw_clear     (w_sw_clear),
    .sw_count_mode(w_sw_count_mode)
```

```
//================================
sensor_unit #(
    .TICK_1US       (TICK_1US),
    .SAMPLE_GUARD_US(DHT_GUARD_US)
) U_SENSOR_UNIT (
    .clk  (clk),
    .reset(w_reset),

    .sr04_start(w_sr04_start),
    .echo      (echo),
//================================
fnd_display #(
    .SCAN_COUNT(FND_SCAN_CNT)
) U_FND_DISPLAY (
    .clk        (clk),
    .reset      (w_reset),
    .mode_sel   (w_mode_sel),
    .disp_mode  (w_disp_mode),
    .msec       (w_msec),
    .sec        (w_sec),
//================================
led_status U_LED_STATUS (
    .mode_sel    (w_mode_sel),
    .wt_edit_sel (w_wt_edit_sel),
    .sw_run_stop (w_sw_run_stop),
    .sr04_valid  (w_sr04_valid),
    .dht_valid   (w_dht_valid),
    .dht_dbg_step(w_dht_dbg),
    .led         (led)
);
```

05 전체를 관통하는 설계 규약 3가지

이 세 가지를 지켰기 때문에 블록으로 나누는 것이 가능했다

01 모든 명령은 1클럭 펄스

btn_debounce 의 o_btn 과 ascii_decoder 의 cmd_* 가 완전히 같은 모양이다.

main_control_unit 에서
is_sw & (btn_r | u_clear)
한 줄로 합쳐진다.

버튼이든 UART 든 상위는 구분할 필요가 없다.

02 단일 클럭 도메인

분주 클럭을 always 감도 리스트에 넣지 않는다.

대신 1클럭 폭 tick 을 만들어
if (tick) …
조건으로 쓴다.

버튼 4개가 있어도 클럭 도메인은 하나뿐이라 CDC 검증 대상이 늘지 않는다.

03 상수는 전부 파라미터로

top 파라미터 8개가 하위 블록까지 그대로 내려간다.

시뮬레이션에서는
top #(.TICK_1US(4), …)
로 줄여서 인스턴스한다.

RTL 을 고치지 않고 같은 코드를 짧게 검증한다.

이 규약이 블록 분할을 가능하게 한 이유

- 명령 모양이 같으니 블록 경계에서 신호를 합치거나 나누는 변환 로직이 필요 없다 — 그냥 OR 로 이어진다.
- 클럭이 하나뿐이니 블록을 떼어내도 클럭 도메인 경계가 새로 생기지 않는다. 테스트벤치가 클럭 하나만 만들면 된다.
- 파라미터가 top 까지 뚫려 있으니, 블록별 테스트벤치도 같은 방식으로 상수만 줄여 빠르게 돌릴 수 있다.

02 왜 7개 블록으로 묶었나

동작은 하나도 바꾸지 않았다. 배선의 위치만 옮겼다

→ 적절한 대답??????

이전 — top 이 모든 것을 알고 있었다

- btn_debounce 인스턴스 4개가 top 에 나란히 놓여 스키매틱에 상자 4개로 흩어졌다.
- 자리수 분리(%10, /10)와 모드별 MUX 가 top 의 always 블록에 있어, 나눗셈 로직이 상자 밖에 떠 있었다.
- rx_data / rx_empty / rx_pop / tx_push / tx_full 이 top 에 널려 있어 UART 내부 배선까지 같이 읽어야 했다.
- 스톱워치와 시계의 출력 두 벌이 top 까지 올라와 있었다.

지금 — top 은 블록 7개를 배선만 한다

- 관련된 인스턴스와 배선을 한 모듈 안으로 내렸다. 스키매틱에 상자 하나로 나온다.
- 블록 내부에서만 쓰는 신호는 블록 밖으로 나가지 않는다. 인터페이스가 좁아졌다.
- top 은 308줄이고 그중 로직은 리셋 동기화 3줄뿐이다.
- 블록마다 포트가 정해지므로 그 블록만 떼어 테스트벤치를 붙일 수 있다.

블록	품고 있는 것	top 에서 사라진 것
btn_unit	btn_debounce ×4	인스턴스 4개
uart_comm	uart_fifo + ascii_decoder + ascii_sender + 송신 타이밍	FIFO 핸드셰이크 배선 6종
main_control_unit	모드 판정 + 명령 병합/게이팅 + FSM 2종	(원래부터 블록)
time_datapath	stopwatch_datapath + watch_datapath + 표시 MUX	출력 두 벌
sensor_unit	sr04_controller + dht11_controller + 값 래치	humidity_reg / temperature_reg
fnd_display	자리수 분리 + 모드 MUX + fnd_controller	always @(*) case 나눗셈
led_status	LED 8개 매핑 (조합)	LED 배선 로직

I 공통 Feedback

사례12 : Text Point 작은 경우

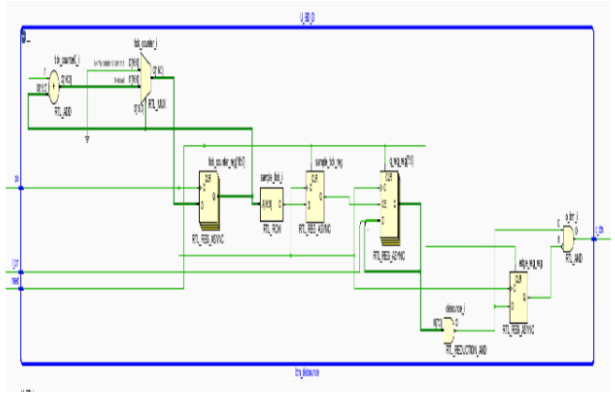
◆ **Feedback**

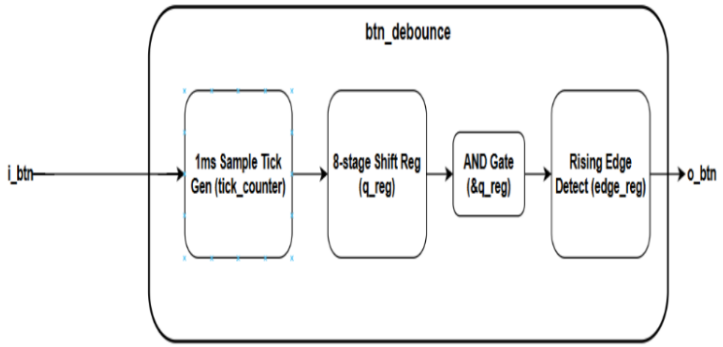
핵심 내용 Font Size ↓ → **가시성 & 가독성 ↓**

Font Size ↑ & 상단 배치 → 내용전달 및 가시성 ↑

btn_debounce — RTL 스키매틱과 블록 다이어그램

입력 버튼은 1ms tick → 8bit shift_register → edge 검출을 거쳐 나간다



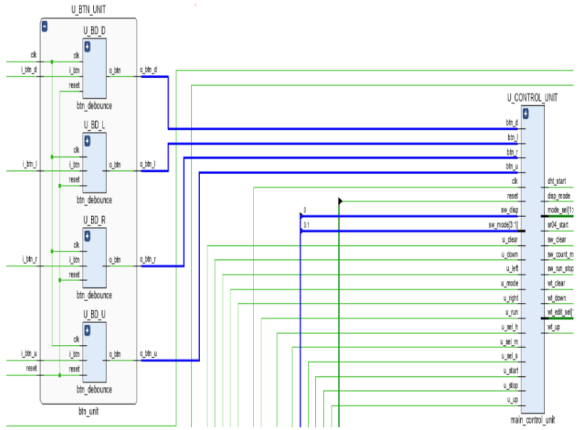


*주석으로 오해할 가능성 ↑

입력 버튼이 들어오면 1ms의 tick_generator를 거쳐서 8bit짜리 shift_register에 쌓이다가 다 차게 되면 edge만 검출시켜 out으로 보낸다.
안정화를 위함

btn_unit → main_control_unit 연결 확인

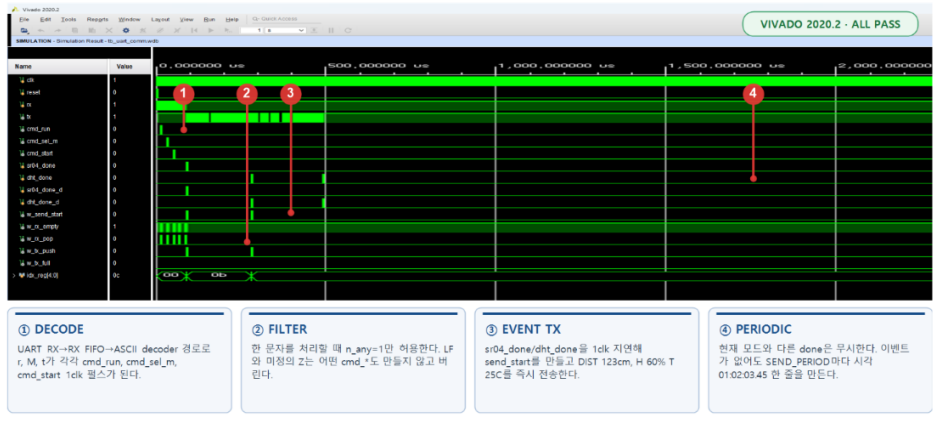
버튼 디바운스 출력이 메인 컨트롤러의 시작·리셋 입력까지 정확히 들어간다



버튼이 메인 컨트롤러에 잘 들어가고 있음을 확인할 수 있습니다 아래의 메인컨트롤러는 각센서에 스타트 여부 와 클락 스탑워치 데이터패스의 리셋 or start 를 결정합니다

23 tb_uart_comm — 확대 파형과 동작 메커니즘

핵심 문자 1바이트는 명령 1펄스로, done/주기 이벤트는 약속된 ASCII 한 줄로 변환



한 클럭 지연을 확대해서 보면, done 엣지가 아니라 그 다음 클럭에 전송이 시작되는 것이 확인된다.

서울기술교육센터

Ⅱ

Ⅰ. 공통 Feedback

Ⅱ. 레퍼런스

Ⅲ. Trouble Shooting

대한상공회의소
서울기술교육센터

Ⅱ 레퍼런스

사례1 : Pin Description 작성

◆ 선정 이유

장표 : Pin I/O 및 Description을 **요약 및 정리에 유용**한 방법, **Datasheet에서 많이 쓰는 방법**

입력 스위치 및 버튼 기능

보드 입력	PC 입력	시계 기능	PC 입력	스톱워치 기능	PC 입력	센서 기능
Btn_U	M	Min_up	m	Up/down count 선택		
Btn_L	H	Hour_up	r	Uart_run	r	Uart_run
Btn_R	S	Sec_up	c	clear		
Btn_D		ampm		laptime		
	s	Uart_stop	s	Uart_stop		
Sw[0]		시간:분,초:밀리초 변환		시간:분,초:밀리초 변환		
Sw[4:1]		sw[1]: watch 출력 선택		Sw[1]:stopwatch 출력 선택		
Led[1:0]		Led[1]:am, led[0]:pm				
reset		reset		reset		

Description of I/O Used

- ❑ Basys 3 보드 사용
  - 보드 내 사용 포트
    - ✔ LED
    - ✔ Pmod connector
    - ✔ fnd
    - ✔ sw
    - ✔ btn
    - ✔ UART port
  - 보드 외 사용 부품
    - ✔ SR-04 센서
    - ✔ DHT11 센서

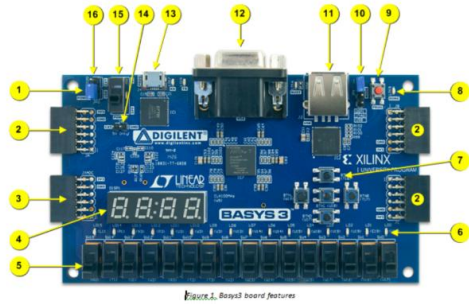


Figure 1. Basys3 board features

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod connector(s)	10	Programming mode jumper
3	Analog signal Pmod connector (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

Ⅱ 레퍼런스

사례2 : Sim. Result 설명 표기 방식

◆ **선정 이유**

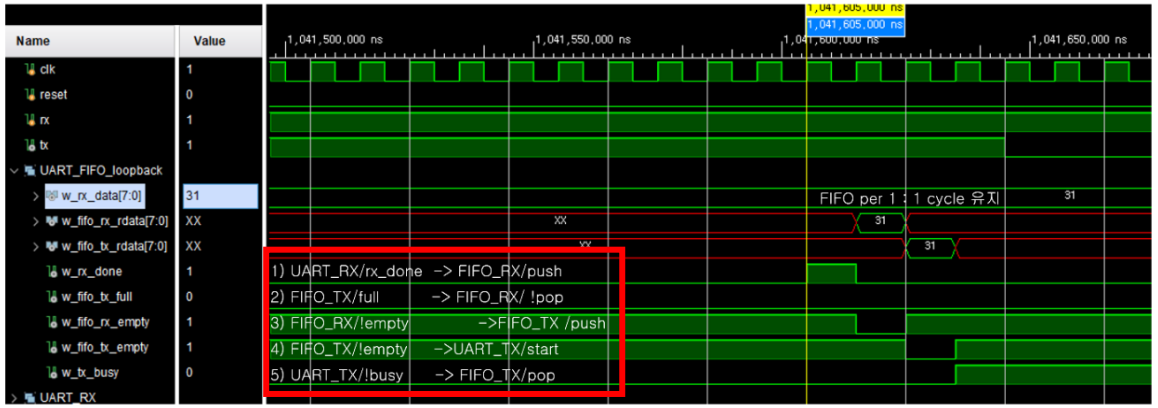
각 Waveform 상태 설명 및 Labeling → **내용의 개연성↑ (순차 및 논리적 구조)**

해당 구간 확대 & Re-captured → **가시성↑**

Sub Module simulation

대한상공회의소

UART_Controller



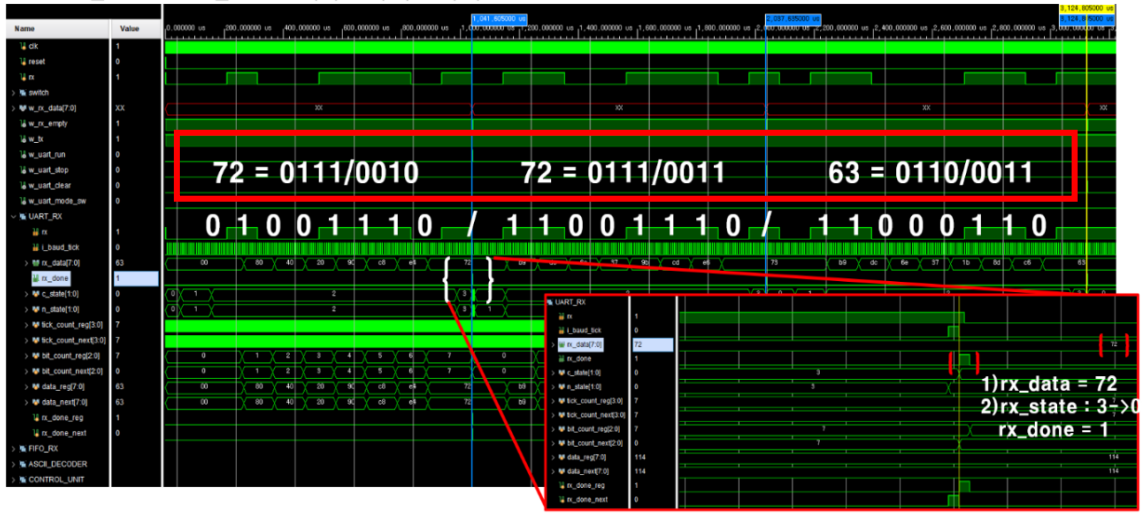
1) UART_RX에서 Done 1 tick 발생 후 FIFO_RX에서 다음 상승 엣지에 Data(31)를 받아 1사이클 유지
2) FIFO_TX가 Full = 0 따라서 FIFO_RX에 Pop이 1로 유지
3) FIFO_RX가 empty가 0이 되어 FIFO_TX는 데이터를 받을 준비가 되어 push가 1
4) FIFO_TX에 데이터가 있으면 empty = 0 UART_TX는 데이터를 받을 준비가 되어 start 1
5) UART_TX가 데이터를 받을 준비가 되어 busy = 0 FIFO_TX는 데이터를 전송 pop 1

Sub Module simulation

대한상공회의

UART Rx → Control_Unit

UART_Rx → FIFO_Rx : h72('r'), 73('s'), 63('c') 데이터 입력



대한상공회의소
서울기술교육센터

Ⅱ 레퍼런스

사례3 : Sim. Result 설명 표기 방식 2

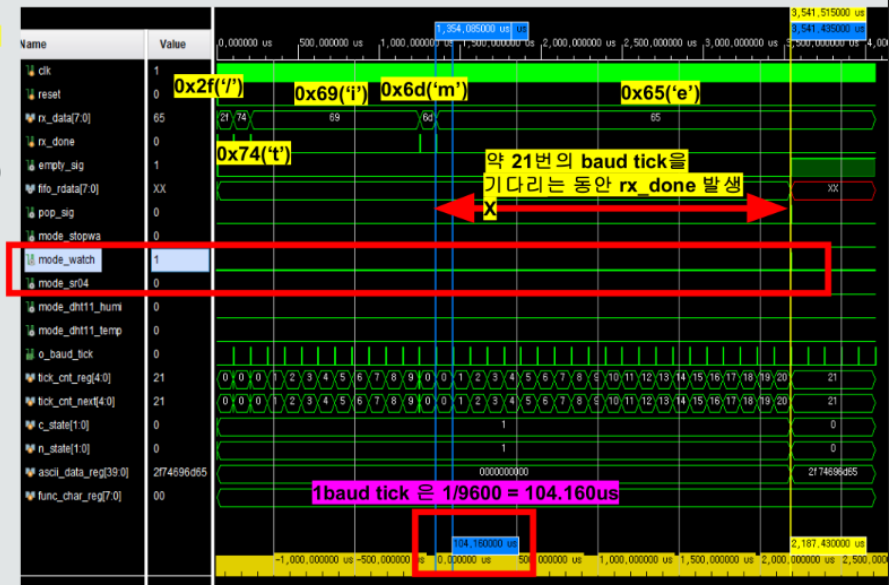
◆ **선정 이유**

Sim. Result 해당 내용 표기 및 Comment → **설명 용이 및 가독성 多, 단계별 분석**

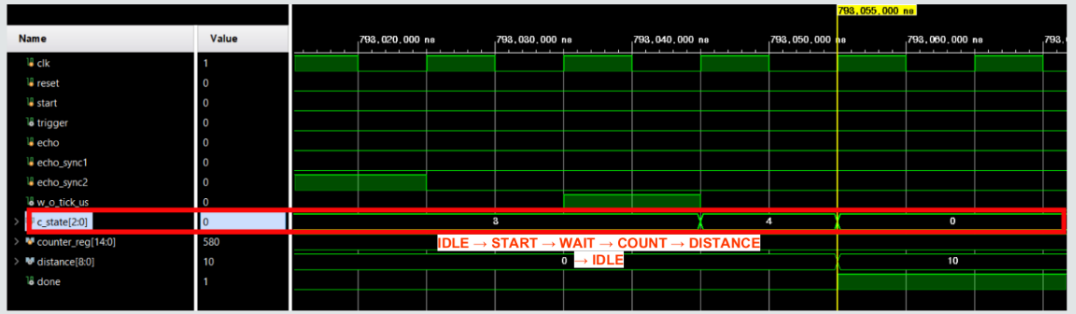
검증 구간의 표시 및 Labeling → **분석 有 가시성↑**

Ascii_Decoder Simulation

- **"/time" 입력 후 일정 시간 지나면 입력 종료로 판단 후 명령어 해석 및 명령어가 가리키는 신호 tick 발생 확인**
- 0x2f('/'), 0x74('t'), 0x69('i') 까지 연속으로 입력 후 10 baud tick 만큼 기다림(tick_cnt 증가)
- tick_cnt가 10이 되는 순간 다시 입력 재개시 rx_done 발생으로 tick_cnt 초기화
- 0x65('e')까지 입력 후 21 baud tick만큼 대기시 자동으로 c_state가 다음상태로 넘어감
- "/time"이 가리키는 mode_watch 신호가 정상적으로 1tick 발생됨



SR04 Simulation (FSM 동작 검증) (Cont'd)



Start 입력에 따라 **SR04 FSM**이 **IDLE → START → WAIT → COUNT → DISTANCE** 순으로 정상적으로 동작하는 것을 확인하였다.

레퍼런스

사례4 : Sim. Result 설명 표기 방식 3

◆ 선정 이유

Case 분류 → **Target 검증 명확, 설명 용이함**

검증 구간의 표시 및 Labeling → **분석 有 가시성↑**

Unit Module Implementation & Verification

Design Verification – Communication Interface Top

Case 3. Key Input Operation

- KEY Table을 함께 도시함.
- 각 KEY에 대한 Decoder출력이 정상적으로 나오는 모습을 볼 수 있음.

Key Table >

Input	Output
C	1000000
F	100000
r	0000001
s	0000010
c	0000011
m	0000001
L	0000010
R	0000011
U	0000001
D	0000010

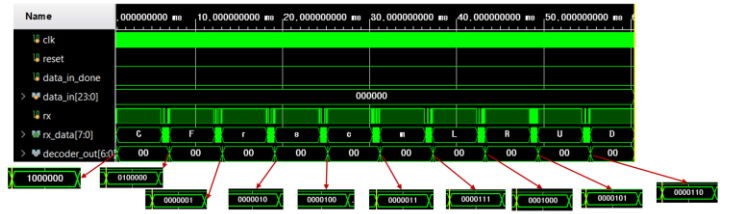


Figure 23. Waveform result of KEY Operation

Unit Module Implementation & Verification

Design Verification – Communication Interface Top

Case 1. /get. Operation



Figure 20. Waveform result of /get. Operation

Unit Module Implementation & Verification

Design Verification – Communication Interface Top

Case 2. /time., /range., /humid., /temp.

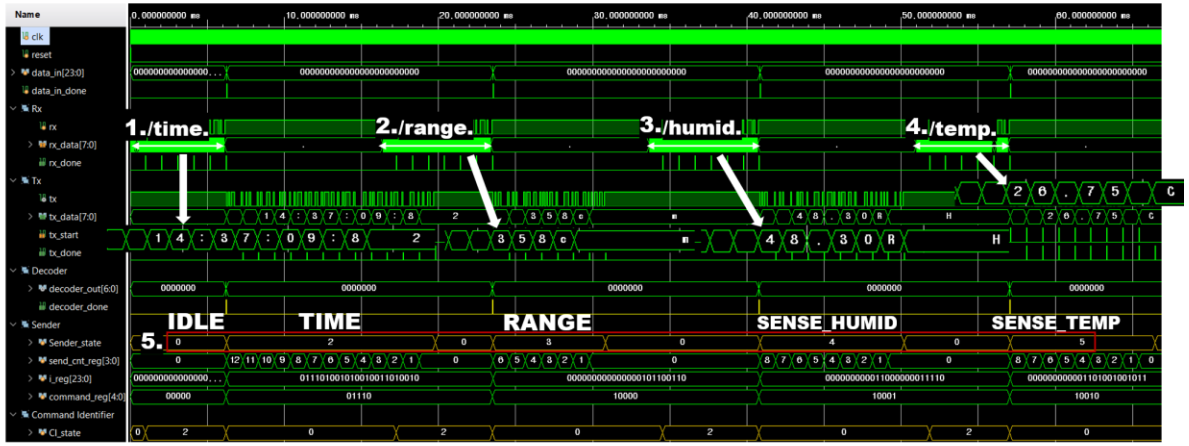


Figure 21. Waveform result of Return Operation

대한상공회의소 서울기술교육센터

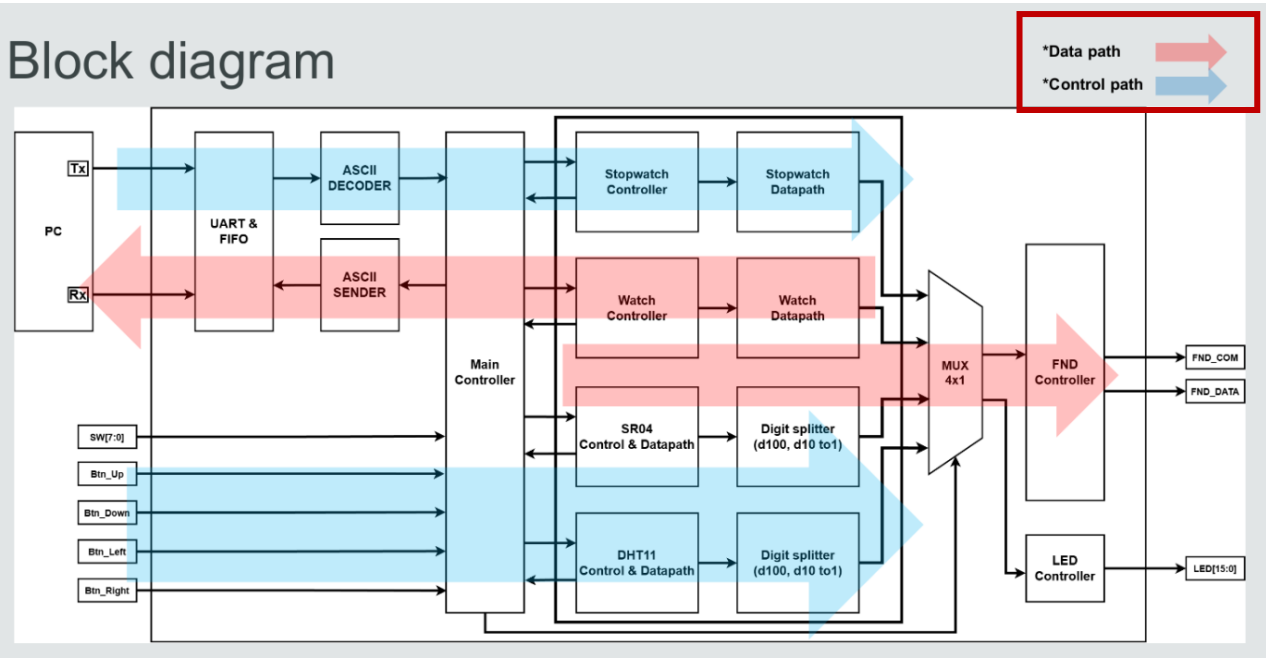
Ⅱ 레퍼런스

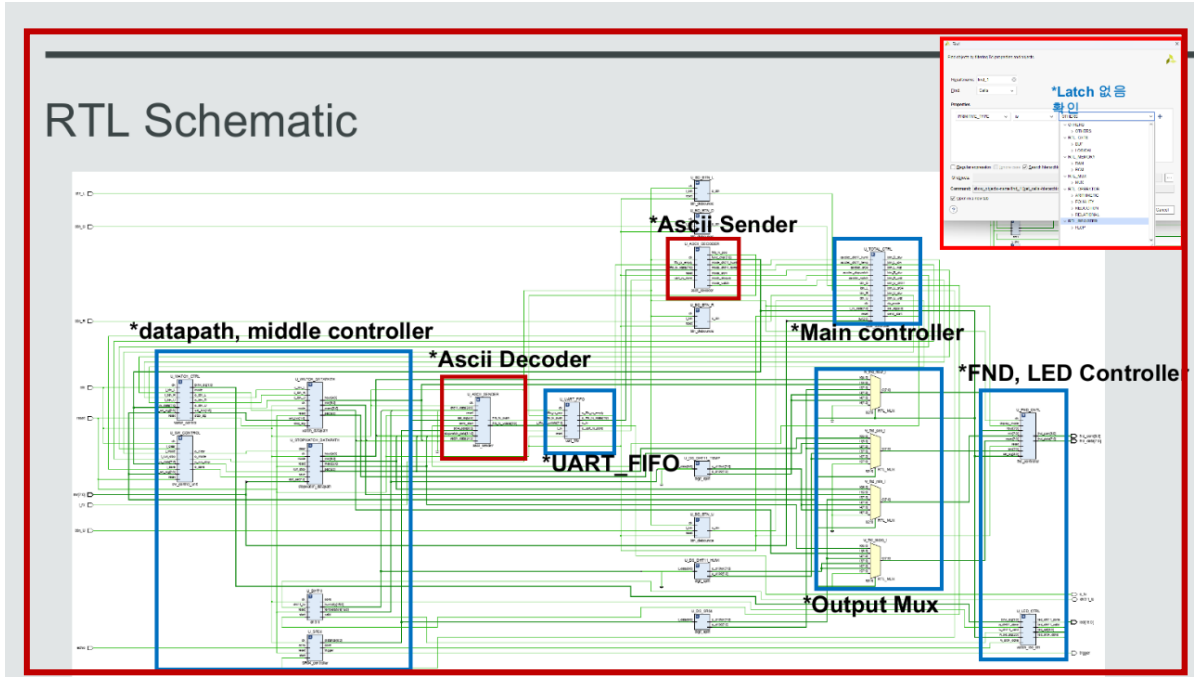
사례5 : Block Diagram 표현 및 Core IP 표기

◆ **선정 이유**

Block Diagram : 필요 IP Block 표현 및 Path 표현 방법 (범례 표현) → **가시성, 가독성 多**

RTL Sch. : Core IP 표시 및 Labeling → **가시성, 가독성 多**





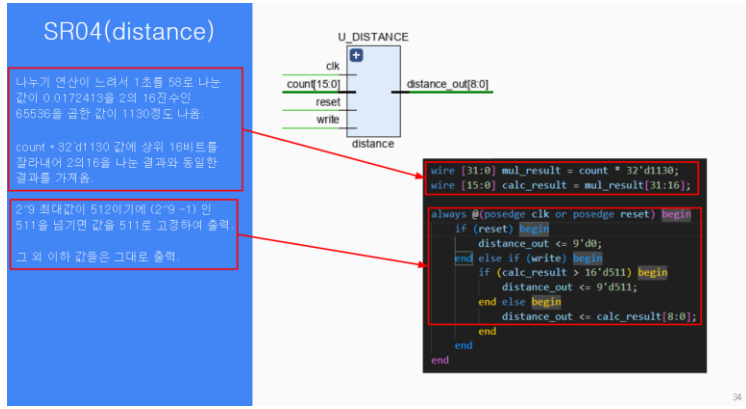
레퍼런스

사례6 : Code & IP Block Matching

◆ **선정 이유**

Code & IP Block Matching → **설명 용이, 가시성, 가독성 ↑**

순차적 설명 및 Square Box 사용 → **가시성 ↑**



Uart_Rx

Baud_tick 신호가 발생 시

- Tick_cnt_reg 가 0이면 값 입력.

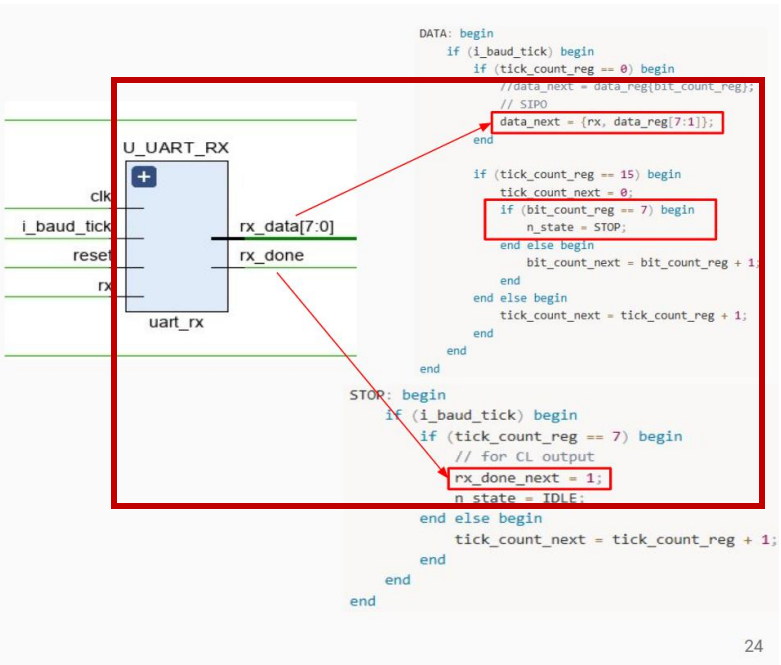
=> 아니면 tick_cnt_reg 1 증가.

- Tick_cnt_reg 가 15이면 tick_cnt_next를 초기화.

=> 아니면 tick_cnt_reg 1 증가

- bit_cnt_reg가 7이면 STOP state.

=> 아니면 bit_cnt_reg 1 증가.



Uart_Tx

- Tick_cnt_next, busy, done 0 / Tx 신호 1

Baud_tick 신호가 발생 시

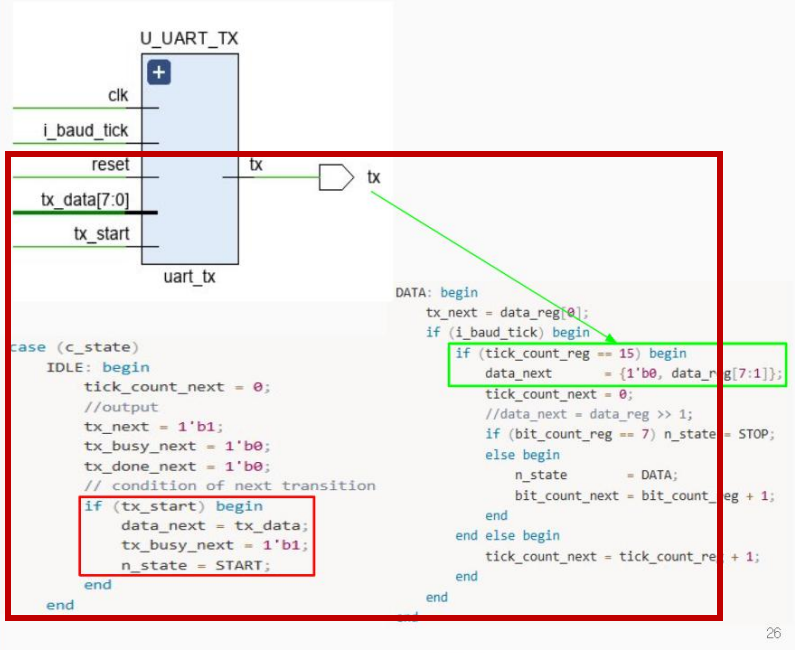
- tx_start 가 1이면 tx_data 넘겨주고, busy를 1로 다음 상태는 START.

- UART는 LSB방식으로 최하위 비트 먼저 연결 및 발송.

- Tick_cnt_reg 가 15면 차례대로 1비트 전송.

- Bit_cnt_reg 가 7이면 다음 상태 STOP.

- 그외 tick_cnt, bit_cnt 1씩 증가.



Ⅱ 레퍼런스

사례7 : 구체적 수치 언급

◆ **선정 이유**

수치 언급 → **간단명료함, 동작 특성을 객관적으로 표현 가능**

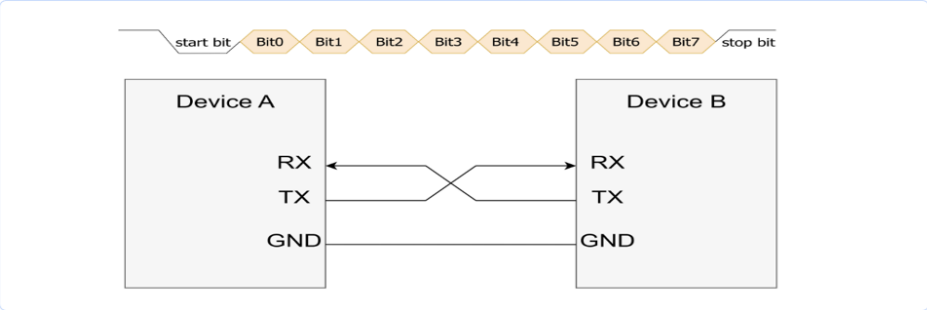
단, **너무 많은 수치 언급은 지양** → 자료 내용의 본질을 흐릴 가능성↑

02 UART RX

UART 통신 개념

클럭 신호 없이 TX/RX 두 선으로 데이터를 주고받는 비동기 직렬 통신(Full-duplex) — 9600bps · 8N1(8 data, No parity, 1 stop)

9600	8	1+1	0
bps (baud rate)	data bits	start bit + stop bit	parity bit



UART 프레임 개념도 — start/data/stop 구조와 두 장치의 RX/TX 교차 연결

03 UART FIFO

FIFO 필요성 & 인터페이스

모듈 간 데이터 생성/소비 속도 차이를 완충하고 데이터 손실을 방지하기 위해 필요하다.

10ns	≈ 1.04ms	≈ 104,000배
1 클럭 주기 (100MHz)	1 byte 수신 시간 (9600bps, 10bit)	클럭 대비 UART 속도 차이

```
module fifo #(
    parameter WIDTH = 2
) (
    input        clk,
    input        reset,
    input        push,
    input        pop,
    input  [7:0] wdata,  //push data
    output [7:0] rdata,  //pop data
    output       full,
    output       empty
);
    wire [WIDTH-1 : 0] w_wptr, w_rptr;

    register_file #(
        .WIDTH(WIDTH)
    ) U_REG_FILE (
        .clk  (clk),
        .waddr(w_wptr),
        .wdata(wdata),
        .we   ((~full&push)),
        .raddr(w_rptr),
        .rdata(rdata)
    );
```

```
    control_unit_ #(
        .WIDTH(WIDTH)
    ) U_CONTROL_UNIT (
        .clk  (clk),
        .reset(reset),
        .push (push),
        .pop  (pop),
        .wptr (w_wptr),
        .rptr (w_rptr),
        .full (full),
        .empty(empty)
    );
```

1. **push / pop / wdata / rdata**
   fifo #(.WIDTH(2))가 RX용·TX용 두 벌로 각각 인스턴스화된다
2. **full / empty**
   각 FIFO의 상태 플래그 — rx_empty=1이면 아직 읽을 데이터 없음, tx_full=1이면 더 못 씀
3. **내부 register_file**
   fifo 모듈 내부의 register_file / control_unit이 실제 저장·포인터 제어를 담당한다

레퍼런스

사례8 : Checklist & Expectation

◆ **선정 이유**

Checklist → 확인 내용 언급, **순차적 확인을 통한 검증 신뢰성↑**

Expectation → 예상 결과 작성, **정상 동작 여부 판단 용이 & 단계별 Debugging 유리**

10 ASCII Sender

시뮬레이션 시나리오

	Input Stimulus OP Check	Checklist	Expectation
순서	입력/동작	확인 내용	예상 결과
1	Watch 12:34:56.78, get_cmd[0]=1	시각 데이터 저장	IDLE → TIME_SEND
2	send_count_reg=0~10	시간 문자 순차 출력	12:34:56.78
3	RETURN → LINE	줄바꿈 출력	CR → LF
4	Distance=263, get_cmd[4]=1	거리 변환 시작	IDLE → CONV_100
5	CONV_100 → CONV_10	백/십/일 자리 분리	2, 6, 3
6	SENSOR_SEND	ASCII 출력	'2' → '6' → '3'
7	RETURN → LINE → IDLE	전송 종료	CR/LF 후 IDLE

10 ASCII Sender

TX FIFO → UART TX 시나리오

순서	입력/동작	확인 내용	예상 결과
1	ASCII_sender tx_push=1, tx_data=8'h31	FIFO_TX가 가득 차지 않았는지 확인	tx_full=0, FIFO_TX에 8'h31 저장
2	uart_tx.tx_busy=0	FIFO_TX pop 조건 성립	FIFO_TX pop=1, rdata=8'h31 출력
3	w_tx_start = ~w_fifo_tx_empty & ~w_tx_busy	FIFO에 데이터가 있으면 자동 전송 시작	uart_tx IDLE→WAIT→START→DATA→STOP
4	직렬 출력	8'h31이 LSB부터 직렬로 tx 핀에 출력	Start→D0~D7(LSB first)→Stop 프레임 확인

ASCII_sender가 만든 문자 하나가 TX FIFO를 거쳐 uart_tx로 넘어가 실제 UART 프레임으로 나가기까지의 전 과정을 검증한 시나리오다.

Ⅱ 레퍼런스

사례9 : 검증을 위한 Modeling

◆ 선정 이유

Ideal Signal이 아닌 **Noise Signal (@Input Signal)** → **검증의 신뢰성 ↑**

Modeling 및 근거 명확 → **논리적 타당성 ↑, 실무 Insight ↑**

10 tb_btn_unit — 검사 항목과 자극 설계

깨끗한 입력만으로는 디바운서를 검증할 수 없다. 일부러 더러운 입력을 준다

```
localparam integer P_SAMPLE = 10;   // 8샘플 = 80클럭

// 30클럭 High / 30클럭 Low 반복.
// 샘플이 10클럭마다이므로 연속 1이 최대 3번
// -> 8번을 못 채워 무시돼야 한다.
task chatter;
    integer k;
    begin
        for (k = 0; k < 5; k = k + 1) begin
            i_i = 1'b1;  repeat (30) @(posedge clk);
            i_i = 1'b0;  repeat (30) @(posedge clk);
        end
        repeat (100) @(posedge clk);
    end
endtask

// 출력 펄스 개수를 세어 "정확히 몇 번" 을 판정한다
always @(posedge clk) begin
    if (o_i) cnt_i = cnt_i + 1;
    ...
end
```

#	검사 항목	판정
1	짧은 채터링은 무시되는가	cnt_i === 0
2	깨끗하게 한 번 누르면	cnt_i === 1
3	2000클럭 눌러도	cnt_i === 1
4	두 번 누르면	cnt_i === 2
5	4채널이 서로 간섭 없는가	R 만 1, 나머지 0
6	리셋 중에는 펄스 없음	cnt_i === 0

Modeling 및 근거

자극을 이렇게 설계한 이유

- 30클럭 떨림은 의도적으로 계산한 값이다. 샘플 주기가 10클럭이므로 연속 1이 최대 3번 — 8번 문턱을 절대 못 넘긴다.
- "펄스가 나왔다" 가 아니라 "정확히 몇 번 나왔다" 를 본다. 개수를 세지 않으면 중복 펄스를 놓친다.
- ③ 길게 누르기는 레벨 신호와 펄스 신호의 차이를 확인하는 항목이다. 여기가 깨지면 상위 카운터가 폭주한다.
- ⑤ 채널 독립은 래퍼를 만들면서 새로 생긴 위험이다. 한 상자에 넣었으니 잘못 배선했을 가능성을 확인한다.
- ⑥ 리셋 중 입력을 High 로 붙잡아 두고 확인한다 — 리셋 해제 순간에 헛펄스가 나가는지 보는 것이다.

03. 시뮬레이션 및 검증 – 검증 시나리오 : step2 SR04

STEP 2. sensor check

DHT11, SR04

Sensor 가 설계 concep에 맞게 outpu을 반환하는지 확인

10cm simualtion				
start=1,10ns	delay negedge trigger	delay 200us	echo=1, 580us	delay 1ms
250cm simualtion				
start=1,10ns	delay negedge trigger	delay 200us	echo=1, 58*250us	

Ⅱ 레퍼런스

사례10 : Table의 적절한 활용

◆ **선정 이유**

각 평가 항목 & Issue List-Up → **정리 유용함, 시스템 단계적 분석 및 가시성↑**

단, PPT 자료 내 장표 비율↑ → 논점을 흐릴 가능성 존재, 적절하게 사용 권장

06 **검증 전략 — 블록마다 전용 테스트벤치**

통합 파형에서 원인을 찾는 대신, 블록을 떼어내 먼저 확인한다

테스트벤치	대상 블록	축소 파라미터	검사 항목
tb_btn_unit	btn_unit	SAMPLE_COUNT 100,000 → 10	채터링 무시 · 1회 펄스 · 길게 눌러도 1회 · 4채널 독립 · 리셋
tb_uart_fifo	uart_fifo	BAUD 9,600 → 625,000 (DIV 10)	수신 1바이트 · 3바이트 순서 · 송신 · 20바이트 무손실
tb_uart_comm	uart_comm	BAUD 625k · SEND_PERIOD 600,000	명령 펄스 · 미매핑 문자 폐기 · 센서 완료 송신 · 주기 송신
tb_main_control_unit	main_control_unit	(없음 — 조합 + 짧은 FSM)	우선순위 인코딩 · 스톱워치 · 시계 · 모드 게이팅
tb_time_datapath	time_datapath	TICK_100HZ 1,000,000 → 4	run/stop · clear · 다운카운트 · 시계 편집 · 출력 선택
tb_sensor_unit	sensor_unit	TICK_1US 100 → 4	SR04 30/100cm · stuck-high 거부 · DHT11 체크섬 성공/실패
tb_fnd_display	fnd_display	SCAN_COUNT 축소	4개 모드 표시 포맷 · 콜론 점멸 · 자리 스캔
tb_led_status	led_status	(클럭 없음 — 조합회로)	모드 one-hot · 편집자리 · valid · DHT 진단 표시
tb_top	top 전체	8개 파라미터 전부 축소	스톱워치 · 시계 · SR04 · DHT11 (`define 으로 개별 on/off)

블록 하나를 고치면 그 블록의 테스트벤치만 다시 돌린다. tb_top 은 블록끼리 이어졌는지만 확인하는 자리다.

55 **해결한 이슈 12건**

통합만으로 드러난 문제 6건 + 동작 버그 6건

#	이슈	원인	조치
1	fnd_controller 두 벌 충돌	시계용·센서용이 같은 이름 다른 내용	4자리 digit + dot 인터페이스로 통합
2	control_unit 이름 충돌	FIFO 내부 제어기와 최상위 CU	fifo_control_unit 으로 개명
3	tick_gen 이름 충돌	sr04 / dht11 이 다른 포트로 각각 정의	tick_gen_1us 하나로 통일
4	clk_div 파생 클럭	분주 클럭을 always 에 직접 물림	전부 클럭 인에이블(tick)로 전환
5	FND · UART 자원 경합	4개 기능이 1벌씩 필요	모드 개념 + fnd_display 안에서 MUX
6	시뮬레이션이 끝나지 않음	실보드 상수 하드코딩	top 파라미터 8개로 전 구간 파라미터화
7	소수점 점멸 무동작	sel 3비트에 4비트 연결 → 상위 비트 잘림	dp 를 세그먼트 비트7 로 직접 제어
8	ASCII 문자 유실	tx_full 확인 없이 push	backpressure 확인 + FIFO 16단
9	DHT11 체크섬 상시 실패	SYNC→DATA 를 레벨 카운트로 판정	하강 엣지 기준 전이로 변경
10	타이밍 위반 −3.32 ns	count / 58 조합 나눗셈	곱셈 + 시프트 근사
11	센서 값이 한 측정 뒤처짐	done 엣지에 래치와 전송이 겹침	uart_comm 에서 done 을 한 클럭 지연
12	센서 무응답 시 시스템 정지	타임아웃 없는 대기 상태	전 상태에 타임아웃 + dbg_step 진단

레퍼런스

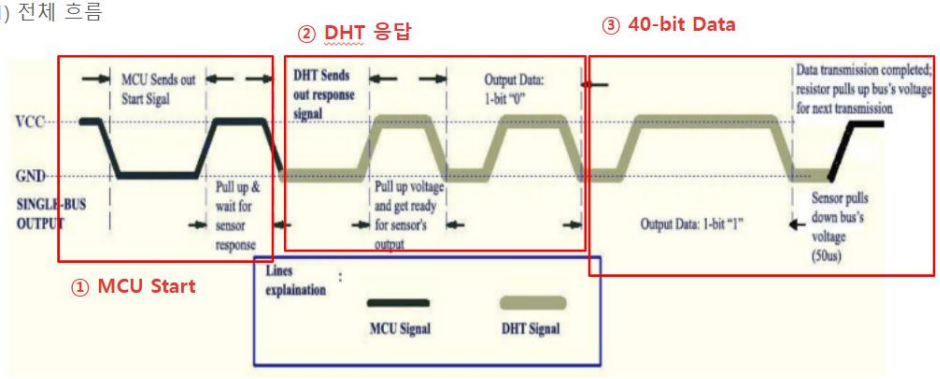
사례11 : Timing Diagram 분석

◆ 선정 이유

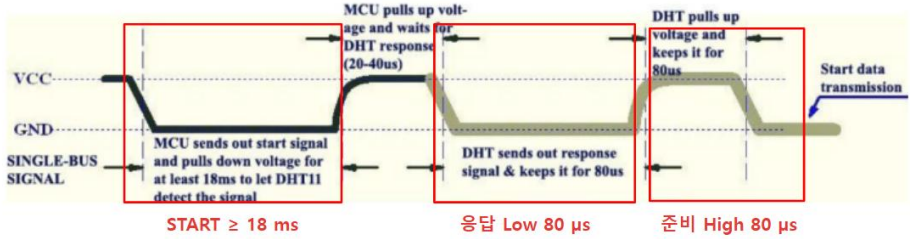
Timing Diagram → **시스템 분석 및 단계적 동작 특성 설명 용이**

단, **Timing Diagram상 기재한 수치를 논리적으로 설명할 줄 알아야 함 (기준 및 시점)**

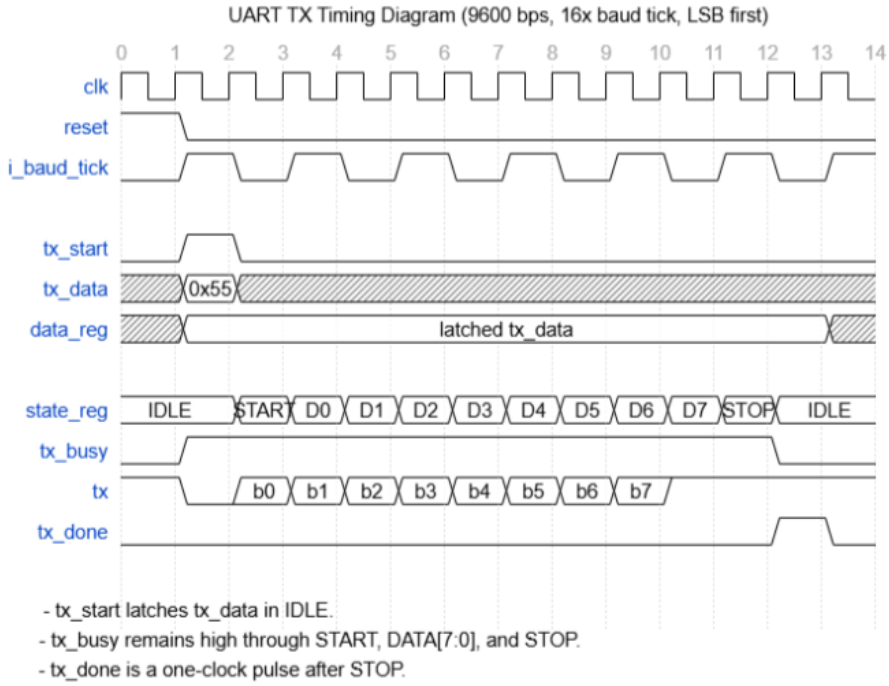
(1) 전체 흐름



(2) START → WAIT → SYNC_L → SYNC_H



응답이 확인되면 DATA_L 상태에서 첫 번째 비트 수신을 시작한다.



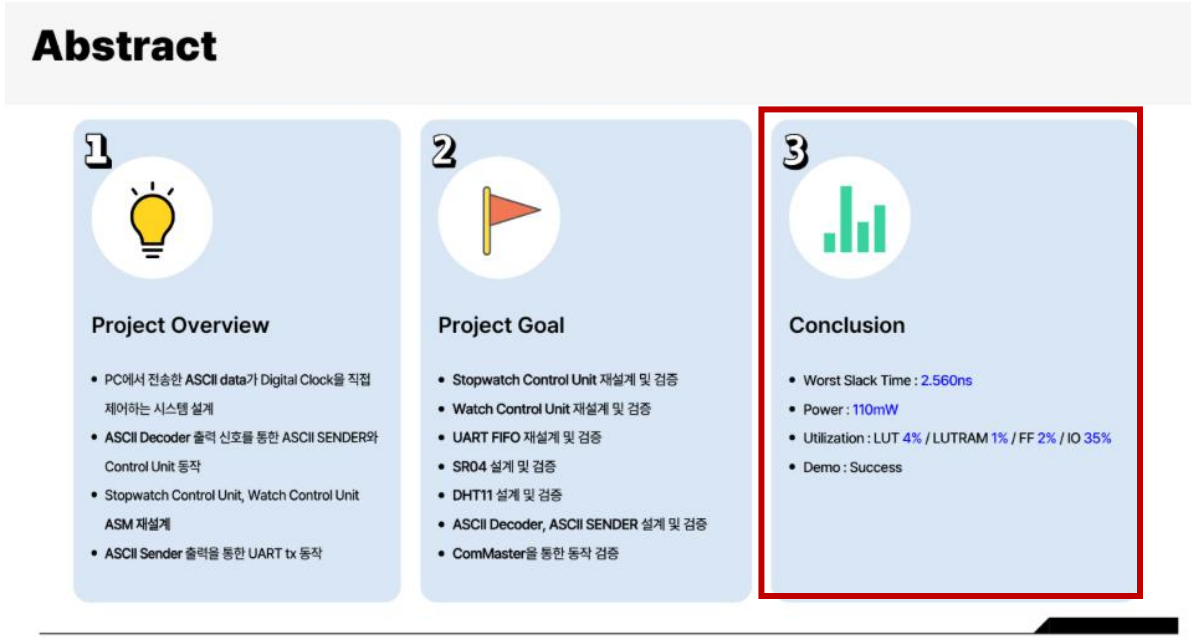
Ⅱ 레퍼런스

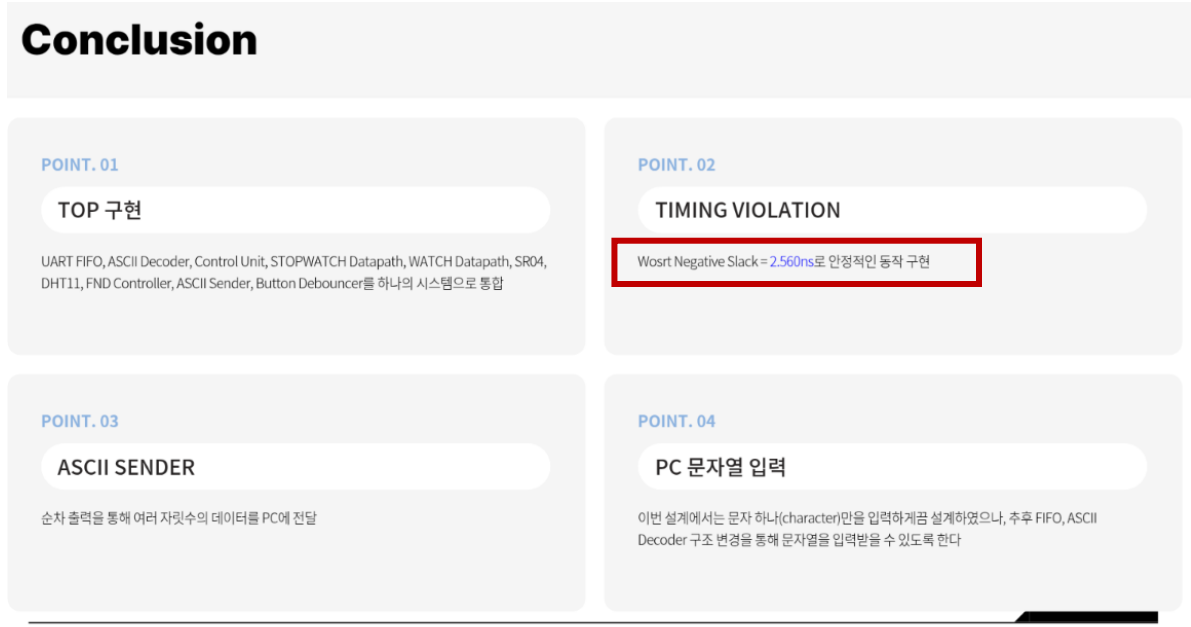
사례12: Abstract 및 Conclusion

◆ **선정 이유**

Abstract → **핵심 요약, 간결함** (가독성 및 가시성↑)

Conclusion → 간결함, **구체적 수치 제시** (검증 신뢰도↑)





Ⅲ

Ⅰ. 공통 Feedback

Ⅱ. 레퍼런스

Ⅲ. Trouble Shooting

대한상공회의소
서울기술교육센터

Trouble Shooting

분석 : 기본 논리

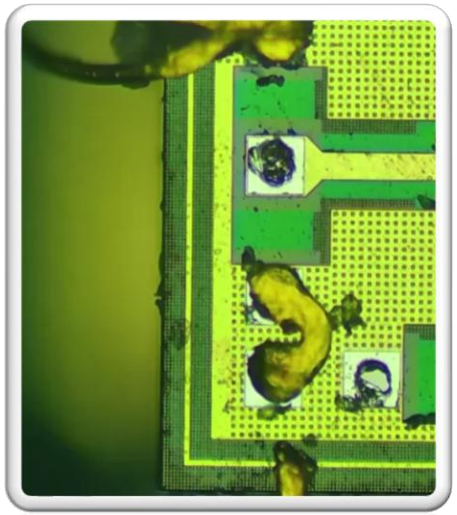
① 현상 — 문제 인식

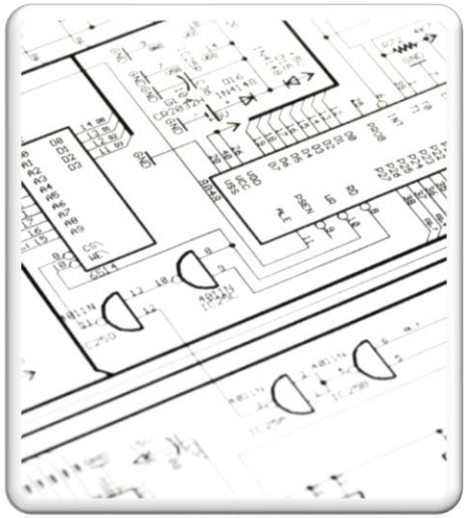
② 원인 — 분석

③ 문제 해결 — A / I

④ 적용 결과 — 비교 및 도출









Trouble Shooting

보강 요청 사례 1

① 현상 → ② 원인 → ③ 문제 해결 → ④ 적용 결과

현재 구현의 한계와 개선 방향

실보드 동작과 코드 리뷰 과정에서 확인된 미완성 지점 3건을 정리함. 기능 자체의 오류는 아니며, 후속 라운드에서 보완 가능한 항목임.

구분	현상 ①	개선 방향 ③
LED 출력	top에서 led = 6'b111111로 하드코딩되어 있어 스위치(sw)와 연동되지 않음.	sw[5:0]과 다시 연결하여 스위치 상태가 LED에 반영되도록 배선 복구.
UART RX 범위	UART 명령(RX) 처리가 watch·stopwatch 모드에 한해서만 확인 안됨. 재확인이 필요함.	모드별 RX 커맨드 매핑을 재확인하고, 누락된 신호를 다시 PC RX로 받을 수 있도록 수정.
top 모듈 구조	유사한 신호·로직이 top 모듈 여러 곳에 분산되어 있어 가독성이 낮음.	역할이 겹치는 신호·always 블록을 통합해 top 모듈을 기능 단위로 재정리.

◆ **Feedback**

→ 현상 자료 누락

→ 원인 누락 (②)

→ 수정 내용 자료 X

→ 적용 결과 누락 (④)

→ 비교 Data 누락

Trouble Shooting

보강 요청 사례 2

1 현상 → 2 원인 → 3 문제 해결 → 4 적용 결과

Troubleshooting

① simulation에서 정상 작동을 함에도 동작 검증 과정에서 지속적인 값의 노이즈가 발생하는 것을 확인

2배씩 값이 변하는 것을 바탕으로 비트밀림 현상임을 파악 후 원인 파악 및 해결

② SYNC state에서 tick count의 천이 조건을 기다리는 과정에서 bit의 밀림이 발생

수정 전
1비트씩 밀림 발생



Troubleshooting
수정 후



◆ **Feedback**

→ 원인 자료 누락

→ 해결 방안 누락 (③)

→ 적용 결과 누락 (④)

→ 비교 결과 자료 보완

Trouble Shooting

보강 요청 사례 3

1 현상 → 2 원인 → 3 문제 해결 → 4 적용 결과

13-1. 결과 · 트러블슈팅

근본 원인은 control unit의 cmd_op 미구분 → signals/target 혼선, cmd_op 연결 수정으로 op 분리 검증

▸ **As-Is → 디버깅 → To-Be**

- **As-Is: RX측 control unit이 decoder 신호를 오해석**
  - cmd_op을 기준으로 삼지 않아 signals(op=0)와 target(op=1)이 혼선 → FPGA에 명령 오전송
- 디버깅: 통합 파형에서 stale signals/target이 op와 무관하게 실행되는 구간 확인
- **To-Be: control unit의 cmd_op 연결 수정**
  - op=0 → signals만, op=1 → target만 해석
  - not-ready 시 kind=1(ERROR) 보호 동작 추가
  - 4종 시나리오 시뮬레이션으로 op 분리 검증

단순 Text 서술 지양

▸ 검증 현황

항목	상태
DHT11 controller	PASS (랜덤100 + ILA)
ASCII decoder	PASS (1byte/멀티)
ASCII encoder	PASS (TB·backpressure)
Control Unit (op 분리)	PASS (cmd_op 수정)
RX 명령 경로	PASS (RX→control)
TX 송신 경로	미검증 (개선 예정)

◆ **Feedback**

→ 현상 자료 누락

→ 원인 자료 누락

→ 문제 해결 자료 누락

→ 적용 결과 누락 (④)

→ 비교 결과 누락

Trouble Shooting

레퍼런스 1

① 현상 → ② 원인 → ③ 문제 해결 → ④ 적용 결과

Trouble shooting 1-wire 링잉 오검출

증상 → 원인

증상 ①

WAIT 에서 센서 응답을 기다리는 동안 가짜 falling 이 잡혀 SYNC_L 로 먼저 넘어간다. 이후 40비트가 밀려 checksum 불일치 또는 타임아웃.

원인 ②

DHT11 은 open-drain + 5K 풀업. 마스터가 START low 를 놓으면 풀업이 배선 용량을 천천히 충전하고, 그 상승 구간에서 문턱 근처 채터링(링잉)이 생긴다. 원시 dht11_io 로 엣지를 잡으면 이게 여러 개의 엣지로 보인다.

→ 해결 → 검증

해결 ③

io_sync[1] 이 io_clean 과 다른 값을 200클럭(2us) 연속 유지할 때만 io_clean 을 바꾼다. rising / falling 검출은 io_clean 에서만 한다.

검증 ④

정상 신호 중 최단 펄스가 26us(0비트 high) 이므로 2us 필터에 잘리지 않는다 (여유 13배). 시뮬 ⑧ 에서 모든 엣지 지연이 정확히 2,020 ns 로 동일 → 비트 폭 보존, 27us / 70us 판정 유지.

◆ Additional Comment

→ 현상~해결 자료 보강

→ Sim. 비교 결과 (Before & After)

→ 적용 Item 자료 추가

→ 기입한 수치 Data

Trouble Shooting

레퍼런스 2

① 현상 → ② 원인 → ③ 문제 해결 → ④ 적용 결과

◆ Additional Comment

→ Sim. **비교 결과 Data 추가** (As is → To be), **순차적 내용 정리 보강** ex) 현상 : ~ , 원인 : ~

Insights

Troubleshooting – Negative Slack Issue

Timing Issue

① ▪ SR04 Control Unit내부 echo_counter -> dis_reg로 넘어가는 과정에서 Negative Slack이 발생.

② ▪ Figure 74. 의 /58연산에 의해 발생하는 것으로 추론.

▪ 이는 SR04 Sensor가Return한 값을 거리로 환산하기 위해 필요함.

▪ 이를 곱연산과 Bit Shifting으로 이를 해결하고자 함.

```
else
    begin
        dis_next = echo_count_reg / 58;
        done = 1;
        n_state = IDLE;
    end
```

Figure 11. Code before Revising

Setup		Hold	
Worst Negative Slack (WNS):	-0.189 ns	Worst Hold Slack (WHS):	0.019 ns
Total Negative Slack (TNS):	-0.401 ns	Total Hold Slack (THS):	0.000 ns
Number of Failing Endpoints:	3	Number of Failing Endpoints:	0
Total Number of Endpoints:	6521	Total Number of Endpoints:	6505

Timing constraints are not met.

Name	Slack	Levels	High Fanout	From
Path 21	-0.189	14	20	U10_SR04/echo_count_reg_reg[10]/C
Path 22	-0.137	14	20	U10_SR04/echo_count_reg_reg[10]/C
Path 23	-0.075	13	20	U10_SR04/echo_count_reg_reg[10]/C

To	Total Delay	Logic Delay
U10_SR04/dis_reg_reg[7]/D	10.143	4.922
U10_SR04/dis_reg_reg[8]/D	10.137	4.916
U10_SR04/dis_reg_reg[3]/D	10.131	4.822

Figure 74. Negative Slack Occurred Timing Report

Insights

Troubleshooting – Negative Slack Issue

Timing Issue

③ ▪ Figure 75. 와 같이 곱연산과Bit Shifting을 통한 나눗셈 연산을 구현.

▪ 해당 코드로 Implementation을 진행한 후 Timing Report를 확인.

④ ▪ Negative Slack이 양수임을 확인.

```
else
    begin
        dis_next = (echo_count_reg * 1130) >> 16;
        done = 1;
        n_state = IDLE;
    end
```

Figure 11. Revised Code to Resolve Negative Slack

Setup		Hold	
Worst Negative Slack (WNS):	2.988 ns	Worst Hold Slack (WHS):	0.027 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	6519	Total Number of Endpoints:	6503

All user specified timing constraints are met.

Figure 75. Timing Report After Code Revising

대한상공회의소 서울기술교육센터

Trouble Shooting

레퍼런스 3

① 현상 → ② 원인 → ③ 문제 해결 → ④ 적용 결과

◆ **Additional Comment**

→ **순차적 내용 정리 보강** ex) 현상 : ~ , 원인 : ~

Insights

Troubleshooting – SR04 Done flag Missing

SR04 Done Flag Missing

①
- SR04 Module 측정 후 Done신호가 누락되는 현상이 발생.
- 이로 인해 Control Unit이 영구 대기 상태로 빠짐.
- Simulation환경에서는 3cm수준을 가정해 Corner Case가 드러나지 않음.

②
- On Board동작 확인 시 Sensor가 항상 최대 거리를 측정하는 상황을 가정해 최대 측정 시간을 초과해 Range측정값을 Return하지 못했음.
- Figure top은 Timeout시 IDLE로 복귀하지만 Done flag발생 로직이 누락됨.

③
- Figure Bottom과 같이 Timeout조건문에 done flag발생문을 추가해 이를 해결하고자 함.

```
RANGING :
    begin
        if(s_echo)
            begin
                if(w_tick)
                    begin
                        echo_count_next = echo_count_reg + 1;
                        if(echo_count_reg == 38000)
                            begin
                                n_state = IDLE;
                                dis_next = 0;
                            end
                    end
            end
    end
```

```
RANGING :
    begin
        if(s_echo)
            begin
                if(w_tick)
                    begin
                        echo_count_next = echo_count_reg + 1;
                        if(echo_count_reg == 38000)
                            begin
                                n_state = IDLE;
                                done = 1;
                                dis_next = 0;
                            end
                    end
            end
    end
```

Figure 76. Revised Code to Generate Done Flag

Insights

Troubleshooting – SR04 Done flag Missing

- 수정된 코드로 ILA를 사용해 측정.
- Echo가 38ms이상인 상황에서 Flag가 발생함.
- Return도 정상적으로 동작함을 확인할 수 있음.

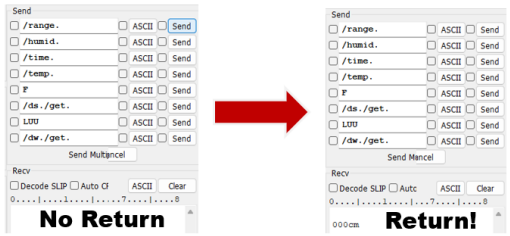


Figure 77. Data Reception (Distance value) Test Results

④



Figure 78. ILA Flag Generation Status

대한상공회의소 서울기술교육센터


Q&A
GO

Thank you